OTA-Shield: A Hardware-Measured Reference Architecture for In-Network OTA Rollout Admission and Bounded Firmware-Attack Detection in Battery Energy Storage Systems

Philip Akekudaga[a,*]

[a] *University of Rhode Island, Kingston, RI, USA*

ARTICLE INFO

Keywords:
Battery energy storage
OTA firmware
programmable data plane
Intel Tofino
intrusion detection
critical infrastructure

ABSTRACT

Battery energy storage systems (BESS) depend on over-the-air (OTA) firmware updates pushed across a fleet of battery management systems (BMS). A coordinated OTA campaign is visible on the network transit path, yet endpoint signature verification cannot see it and current BESS defenses do not use this signal. We present OTA-Shield, a hardware-measured reference architecture that admits authorized firmware rollouts and operator rollbacks and detects a bounded family of OTA firmware attacks on a cleartext reference MQTT channel. An Intel Tofino 1 pipeline parses the OTA header and tracks per-BMS state in the data plane; a control-plane Rollout-Authorization Table (RAT) and a two-stage arbiter check observed events against the operator's rollout schedule, admitting scheduled rollouts and operator rollbacks instead of dropping them.

On a 50-unit emulated BMS testbed the detector classifies the designed attack family at $F_1 = 0.996$ with no false alarm on benign traffic (0 of 600 events; Clopper–Pearson 95% upper bound 0.50%), at a median end-to-end latency of 34.0 ms, well inside the minutes-to-hours window of an OTA campaign. A CPU port of the same rule-and-arbiter logic matches detection quality exactly, so the contribution is architecture and resource placement rather than switch line rate, which we do not measure. We report the boundaries as well as the successes: detection needs a cleartext segment after TLS or VPN termination, an attacker who holds a RAT-authorized source and version and stays inside the rollout envelope is out of scope, and against an adaptive adversary who reshapes cadence and volume coverage falls to 1.7% of attack events.

1. Introduction

Battery energy storage is becoming a critical grid asset. Compound annual growth in installed capacity currently runs at 30 to 45% in major markets [**?**]. Every installation depends on over-the-air (OTA) firmware updates pushed to its battery management system (BMS) fleet, typically over MQTT or a proprietary equivalent. When that channel is compromised, an attacker can flash malicious firmware to dozens or hundreds of BMS units at once. Fleet-wide BMS misbehavior has documented grid-impact pathways: modelled frequency-response loss [**?**], thermal runaway as documented in the McMicken 2019 and Moss Landing 2024 incidents (neither attributed to OTA compromise), and supply withdrawal during peak demand [**?**]. None of these incidents is attributed to OTA compromise. Quantitative grid-impact study is out of scope; the documented attack surface and the documented impact pathways are independently sufficient motivation.

Current practice secures the endpoint, not the transit path. SUIT (RFC 9019) [**?**] and Uptane [**?**] prevent an individual device from installing unsigned firmware, but neither constrains the fleet fanout rate of validly signed images. Uptane's release-counter and director-repository design constrain which image a target may install, not the fanout pattern used to reach the fleet, so in-transit detection is complementary rather than a replacement. IEEE 2030.5-2018 mandates mutual TLS but does not require online OCSP/CRL; field deployments often omit revocation, and Sandia/SunSpec implementation guidance leaves revocation handling to the deploying aggregator [**? ?**]. A stolen certificate retains operational validity until that aggregator's policy revokes it, so revocation latency is the relevant attack window. CPU-based signature IDSes such as Suricata and Snort 3 were built around stateless or short-horizon rules, and the long-horizon per-BMS state needed to correlate an OTA campaign is awkward to express in their rule languages [5**?**].

This leaves a specific gap. A coordinated OTA campaign is visible on the network transit path, where the fleet-fanout pattern is exposed, yet it is invisible to endpoint signature verification, which sees each device in isolation. Existing BESS security research either models attack surfaces without proposing a detector [**? ?**], or places detection on the device through side-channel or telemetry ML [**?**]. None of these approaches observe the network transit path, which is where coordinated campaigns become visible.

Programmable switches provide a per-packet vantage point for that observation. An Intel Tofino 1 pipeline can parse MQTT, extract an OTA header, maintain per-flow counters, and fire detection rules inside the

*Corresponding author.
akekulip@gmail.com (P. Akekudaga)

ASIC forwarding path [**?**]. We use the Tofino 1 for its programmability, not for a throughput claim. Earlier work applied these primitives to volumetric DDoS [**? ?**] and generic per-flow ML [**? ?**], but we are not aware of any prior work targeting application-layer OTA firmware attacks on BESS infrastructure.

This paper describes OTA-Shield, an OTA anomaly detector for BESS fleets. Rule checks run in a Tofino 1 programmable data plane and the admission logic in a small control-plane arbiter. Four rules compile into the data plane (R2 unauthorized source, R4 oversize, R5 unauthorized-source fanout telemetry gated behind R2, R6 per-BMS version monotonicity), and the arbiter consults a Rollout-Authorization Table so that scheduled rollouts and operator rollbacks are admitted instead of dropped. Only the terminal R2 drop is resolved entirely in-switch; R4 and R6 (and R1 where it applies) raise a HOLD that the controller resolves in the loop. We use the Tofino 1 for early, fail-closed filtering and to compile the rule set into a fixed resource budget. The terminal attack rules (R2, R4, R6) need no control-plane help to *detect*; the arbiter handles the harder case, separating an authorized rollback or re-push from a rollback attack or a replay, which we evaluate directly in E19 and E12. We present OTA-Shield as a hardware-measured reference design for in-network OTA detection on Tofino 1, evaluated end-to-end on a 50-unit emulated BMS testbed with a single reference MQTT profile. Two scope boundaries apply throughout. First, OTA-Shield reads cleartext OTA fields, so it must sit on a plaintext segment after TLS or VPN termination. Second, the deployed fleet-fanout rule (R5) counts only unauthorized-source fanout, so an attacker who holds a RAT-authorized source identity and firmware version and operates inside the authorized rollout envelope is outside the scope of R5-based detection. Coordinated fanout of validly signed firmware from a stolen but authorized key is thus the motivating case, which we report as out of scope rather than as a result: on the deployed build the delivered detection contribution is R6 version-monotonicity (catching a signed-but-older image that endpoint verification accepts) together with the arbiter's false-positive-free admission of authorized rollouts and rollbacks. The evaluation isolates the arbiter's contribution, reports each rule's blind zone under mimicry, compares against Suricata and Zeek on identical traffic, and measures parser portability bounds, scoping all claims to the reference profile measured here. Our contributions are:

- A **two-stage RAT arbiter** (Gate A, Gate B) that admits authorized rollouts and rollbacks without false positives. Its value is admission (E12, E19), not attack precision: on the deployed gated build, disabling it leaves attack precision unchanged (E4).
- An **OTA-aware P4 pipeline** on Tofino 1 that extracts a 20-byte OTA header and runs four bounded rules: R2 (unauthorized source), R4 (oversize), R5 (fleet-fanout telemetry, gated behind the terminal R2 drop), and a per-BMS version-monotonicity register (R6). R1 (rapid replay) is cadence-gated and compiled out below $\tau_{\mathrm{R1}} = 14\,400$ s [**?**].
- **Operator-policy thresholds** from a 90-minute baseline trace: R4 at measured percentiles (P99+$k\sigma$ [**? ?**]), with R5 and R1 as compile-time constants informed by the trace.
- A **hardware-measured evaluation** on a 50-unit emulated BMS testbed: ablation, adversarial sweeps, benign-rollout stress, held-out mimicry, override-table capacity, parser portability, and a Suricata/Zeek baseline.
- A **deployment-boundary characterization** of programmable in-network OTA security: where the design holds, and where adaptive mimicry (1.7%, E21), brokered MQTT (E20), TCP segmentation (E23), and parser variation (E18) require an extension. We report this as a measured boundary, not a robustness claim.

Scope and transferability. The OTA channel used throughout is a reference channel (MQTT over TCP/1883 with a 20-byte application-level OTA header), not a specific vendor implementation. What transfers to other profiles is the construction: a PHV-aligned OTA-header parse, a fleet-cardinality counter (R5), a per-BMS version high-water register (R6), and a two-stage RAT arbiter (Gate A, Gate B). Each of these is implementable on any $\mathrm{P4}_{16}$ target with equivalent register primitives. What does not transfer verbatim are the numerical thresholds and the MQTT framing: R1's interval is tied to per-BMS update cadence, and the parser is bound to the reference framing measured in E18. The throughput measurement is scapy-limited to $\approx$2,000 pps on our generator and is reported as a controller digest-channel sustainable-rate lower bound, not a switch line-rate proof; §7 discusses the further limits this entails.

2. Background and Threat Model

2.1. BESS OTA architecture

A grid-connected BESS installation comprises 10–500 battery modules, each governed by a BMS controller that regulates charging, discharging, and thermal protection [**?**]. An OTA server operated by the battery vendor or system integrator pushes firmware images to the BMS fleet over an IP network. Commercial vendors vary in how they implement this channel: some use proprietary HTTPS endpoints, some use MQTT, and some use vendor-specific protocols layered on IEEE 2030.5 [**? ?**].

For the purposes of this work we adopt a reference OTA channel rather than a specific vendor implementation. The channel uses MQTT on TCP port 1883 for firmware distribution, with each PUBLISH carrying a 20-byte application-level OTA header (magic signature, firmware version, advertised size, and a hash hint). This header format is not proprietary to any one deployment; it captures the minimum semantic information the network needs to recognize an OTA flow. We treat the reference channel as a plausible synthetic deployment target, not as a universal model of real BESS OTA practice. The techniques we develop (protocol-aware extraction in the data plane, fleet-cardinality tracking, and policy-aware arbitration) transfer to any channel that exposes equivalent semantic fields; ports and header layouts are configuration.

2.2. Regulatory gap

NERC CIP standards govern cybersecurity for bulk electric system assets, yet smaller distributed energy resources (DERs), including most BESS installations under 75 MVA, fall outside CIP's mandatory scope [**?**]. The NERC RSTC white paper on DER cybersecurity acknowledges this gap and recommends voluntary controls, but compliance remains sparse [**?**]. IEEE 2030.5-2018 mandates TLS mutual authentication for DER aggregator communications. The standard does not specify a revocation profile that operators can use to invalidate a stolen device certificate on the OTA path inside operational SLA windows, and Sandia/SunSpec implementation guidance leaves revocation handling to the deploying aggregator [**?**]; absent such a profile, a compromised certificate persists across the deployment until manual rotation [**?**]. NFPA 855 references UL 2900-1 for network-connectable product security, but its primary concern is fire protection, not network-transit defense [**?**]. IEC 62443 provides a defense-in-depth framework for industrial automation, yet prescribes no specific mechanism for detecting coordinated firmware distribution campaigns [**?**].

2.3. Threat model and attack taxonomy

The adversary has access to the OTA distribution channel and a valid or stolen signing key, consistent with documented supply-chain compromises [**?** **?** **?** **?**]. Endpoints are assumed to verify signatures; Table 1 states the full capability, vantage, and scope boundaries.

Within this model five data-plane-observable attack patterns motivate our detection rules, with a separate control-plane concern (OTA-header protocol anomalies) handled by the Python controller rather than in the pipeline:

A1 Fleet fanout Firmware pushed to an anomalously large number of distinct BMS units within a short window, exceeding any authorized rollout pace. Endpoint verification cannot see this pattern. Detected by rule R5 only when the fanout originates from an unauthorized source; authorized-source fanout inside the RAT envelope is out of scope (§6.4).

A2 Unauthorized source Firmware originates from an IP not listed in the operator's Rollout-Authorization Table. Detected by rule R2.

A3 Oversized payload A single MQTT session carries more bytes than the maximum legitimate firmware image size. Detected by rule R4. R4 uses a P4 range-match against a compile-time size table and therefore exposes a bounded dead zone between the legitimate maximum and the next table boundary (measured at 64 KiB, covering 2.0–2.0625 MiB; §6.5) through which an attacker can slip an oversize but in-dead-zone payload.

A4 Rapid replay The same BMS receives a second firmware push within a window shorter than any legitimate maintenance cycle (typically 4–24 hours). Intended to be detected by rule R1, which is effective only when per-BMS update cadence is well below the 14 400 s threshold; see §6.5.

A5 Version rollback A signed-but-older firmware image is pushed to the fleet to revert devices to a vulnerable prior state. Endpoint signature verification accepts the image as long as the signing chain is still valid. OTA-Shield addresses this with rule R6, a version-monotonicity check against a per-BMS high-water mark; authorized downgrades are expressed as an explicit `rollback_window` in the RAT and handled by the rollback gate (§4.4.2).

Table 1
What can go wrong in BESS OTA distribution, and where each threat is caught.

Assumption	Detail
Attacker capability	On-path access to the OTA channel (server compromise, stolen TLS cert, or aggregation-point injection) with a valid or stolen signing key.
Attacker goal	Distribute malicious firmware to a BMS fleet quickly enough to affect grid operation before operator intervention.
Defender vantage	Programmable switch on the OTA distribution path (in transit, not on endpoint).
Out of scope	Signing-key cryptanalysis; endpoint exploitation independent of OTA; full TCP reassembly; encrypted-MQTT variants that hide the OTA header from on-path inspection (IEEE 2030.5 mandates TLS; OTA-Shield requires a cleartext segment after TLS/VPN termination); adversaries with valid RAT-whitelisted source IPs and firmware versions operating within authorized rollout parameters (R5 Bloom inserts are gated on unauthorized-source flag, so compliant behavior from a stolen-credential attacker passes R5 undetected).

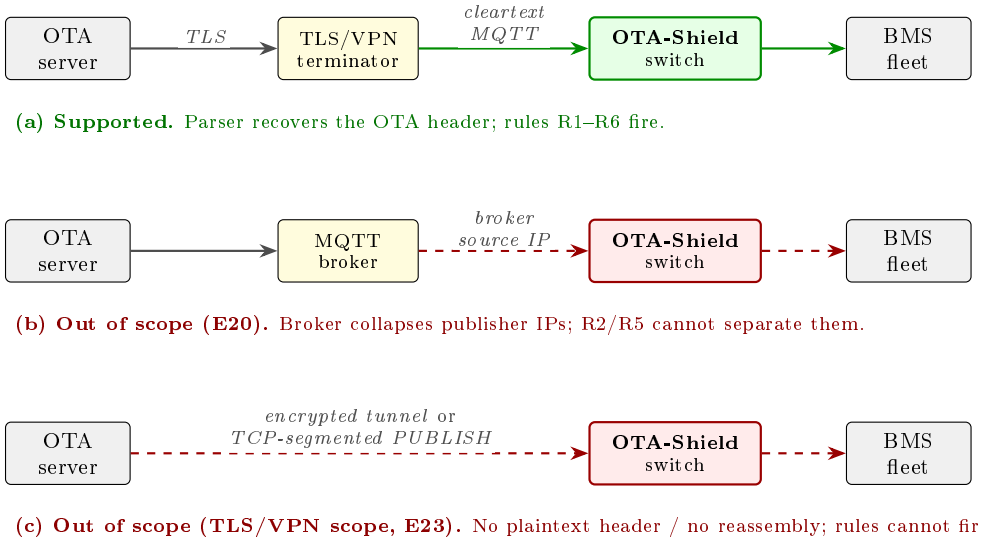


Figure 1: Deployment scope, fixed by the threat model. OTA-Shield is supported only on a cleartext MQTT segment after TLS or VPN termination (a), where the parser recovers the OTA header and rules R1–R6 fire. Two paths are outside the evaluated build: (b) a brokered path, where the broker rewrites every PUBLISH to its own source IP and the source-based rules can no longer separate publishers (evaluated as a negative control in E20, §7.2); and (c) an encrypted tunnel that reaches the switch intact, or a PUBLISH split across TCP segments, where no single packet presents a parseable header and the data plane performs no reassembly (E23, §6.5). Dashed red paths mark unsupported placements, not demonstrated capability.

The rules are numbered non-contiguously because R3 was reserved for a protocol-anomaly rule that we moved to the controller's manifest check. The data plane carries four mandatory rules (R2, R4, R5, R6) plus one optional rule (R1), together with one control-plane check. R1, R4, and R5 are the three threshold-bearing rules, R2 is identity-based, and R6 is monotonicity-based. R1 is enabled only when per-BMS update cadence is slower than $\tau_{R1} = 14\,400$ s; under faster cadences it is compiled out, leaving R2/R4/R5/R6 as the operational rule set.

This threat model also fixes where the detector can sit. OTA-Shield observes an OTA campaign only on a cleartext segment where it can recover per-packet source identity and OTA-header fields. Fig. 1 sets out the three placement paths this implies: a supported direct path after TLS/VPN termination, and two paths the evaluated build does not cover, a brokered path that collapses publisher source addresses and an encrypted or TCP-segmented path that hides the header. The two unsupported paths are characterized later as negative controls (E20, §7.2; E23, §6.5); we introduce them here so the scope is explicit before the design.

3. Related Work

We organize prior work by what each line of research can express, the assumptions it makes, and the limitation that leaves the OTA fleet-coordination problem open.

In-network security on programmable switches. Programmable-switch security work can enforce per-packet policy at line rate, but the published targets are volumetric or generic. Most P4/Tofino work targets volumetric DDoS: Poseidon compiles declarative defenses [**?**], Jaqen reaches 380 Gb/s with sketch-based mitigation [**?**], Patronum adds per-flow state [**?**], AlSabeh et al. run entropy-based DPI [**?**], and ACC-Turbo handles pulse-wave traffic [**?**]. Beyond DDoS, Kang et al. enforce context-aware enterprise access-control policies (BYOD) in the data plane without a control-plane round-trip [**?**], and Xing et al. mitigate network covert channels at line rate while preserving TCP performance [**?**]; both confirm that the programmability benefit reaches policy-enforcement and behavioral-anomaly problems beyond volumetric attacks. A parallel line pushes flow-level machine learning into the data plane [**? ? ?**]. The closest methodological neighbor is Bui et al. [**?**], which parses MQTT in P4 on the BMv2 software target and enforces topic-prefix authorization with 99.8% accuracy and sub-ms latency. Its assumption set differs from ours on platform, domain, threat model, policy model, stateful scope, and evaluation rigor; Table 2 marks the six axes. None of this prior work targets OTA firmware attacks on grid infrastructure.

OTA firmware security and BESS prior work. This line of work secures the firmware after it arrives, which leaves fleet-wide coordination during distribution

Table 2
Axis-by-axis comparison with Bui et al. [**?**].

Axis	Bui et al.	OTA-Shield
Target platform	BMv2 software reference; no MAU budget or PHV layout constraints.	Tofino 1 hardware; reports measured layout against the 12-stage MAU budget, 48-byte digest quantum, and register restrictions (§4).
Target domain	Generic edge-IoT MQTT anomalies.	BESS firmware distribution on a grid-operator path (§2).
Threat model	Endpoint compromise and topic-ACL abuse.	Channel compromise with valid signing material and authorized topics, fleet-coordinated campaigns.
Policy model	Stateless topic-prefix authorization.	RAT with Gate A and Gate B arbitration (Alg. 1) over per-BMS rollout context.
Stateful scope	Session-level flow state.	Per-BMS version high-water (R6) and Bloom-gated fleet-fanout counter (R5).
Evaluation rigor	99.8% enforcement on software-timed BMv2.	Hardware latency (median ≈34.0 ms, p95 ≈46.6 ms), bootstrap F1 CIs over 20 stochastic trials, ablation (§6.4), adversarial-boundary sweep (§6.5), Suricata/Zeek baseline on identical traffic (§6.6).

invisible. Firmware-update attacks on IoT devices are well documented [**? ?**]; AoT shows replay and rollback on real devices [**?**]; SUIT [**?**] and Uptane [**?**] standardize cryptographic signing; and DDoSViT [**?**] defends the OTA channel from disruption. All of these verify firmware after delivery, so fleet-wide coordination during distribution is invisible to them. On the BESS side, Öhrström et al. give the first cloud-BESS attack reference model [**?**], Trevizan et al. catalogue BESS risks and controls [**?**], Culler et al. observe active scanning of exposed BMS interfaces [**?**], and GAN/ensemble detectors run on-device over telemetry [**?**]. The NERC RSTC white paper recommends in-network visibility as a compensating control for small DERs outside mandatory CIP scope; OTA-Shield fills that gap on switching silicon.

Threshold derivation and CPU-based IDS baselines. A CPU-based signature IDS has the throughput but not the arbitration needed for this problem. IDS anomaly thresholds are usually set from measured baseline distributions rather than expert heuristic [**? ? ?**]; we follow the same P99+$k\sigma$ pattern on a 90-minute on-testbed trace (§6). Suricata and Snort remain the reference open-source ICS IDS [**?**], and Suricata reaches tens of Gb/s via AF_PACKET [5], well above the OTA-channel loads here. The meaningful question is therefore not throughput but expressiveness: whether the rule language can capture stateful, RAT-aware arbitration. §6 answers that directly on identical traffic.

4. System Design

OTA-Shield splits detection between a Tofino 1 data plane that enforces per-packet rules inside the ASIC pipeline and a Python control plane that arbitrates ambiguous detections against a Rollout-Authorization Table (RAT). This subsection describes what each component does, why it is needed, and the constraint it addresses; the hardware realization follows in §5.

4.1. Architecture overview

Fig. 2 shows the split. The data plane carries the always-on, per-packet work: it parses the OTA header and evaluates the detection rules at forwarding speed, where a coordinated campaign's fanout shape is exposed. The control plane carries the slower, policy-aware work: it holds the rollout schedule and decides whether a flagged event belongs to an authorized rollout or to an attack. The split is needed because the campaign-versus-attack decision requires state (the rollout schedule) and matching logic that do not fit the per-packet ASIC budget, while the cheap campaign-shape signal must run on every packet without a control-plane round-trip.

4.2. Operational placement and responsibilities

OTA-Shield sits on the cleartext OTA segment between an upstream TLS/VPN terminator and the BMS fleet, and it divides work among the parties already present in a BESS OTA rollout. Endpoint firmware signing remains in place and complementary; OTA-Shield adds a network-side admission and detection layer that the endpoints cannot see. Table 3 states who owns each step.

4.3. Detection rules

Each rule answers one attack pattern from §2. R2 (unauthorized source) is an identity check on the firmware source IP against the operator's allow-list; its input is the IPv4 source address and its output is a terminal fire when the address is absent, implementing a fail-closed policy. R4 (oversize) compares the cumulative session byte count against the maximum legitimate firmware size; it is terminal because no authorized image exceeds that bound. R5 (fleet fanout) counts the distinct destination BMS addresses an *unauthorized* source reaches inside a 60-second window; its counter is gated on the unauthorized-source flag (`r2_fired`), so on the deployed build it charges only behind the terminal R2 drop and functions as a fanout telemetry signal rather than an independent verdict (§6.4). R6 (version rollback) compares the OTA-header version against a

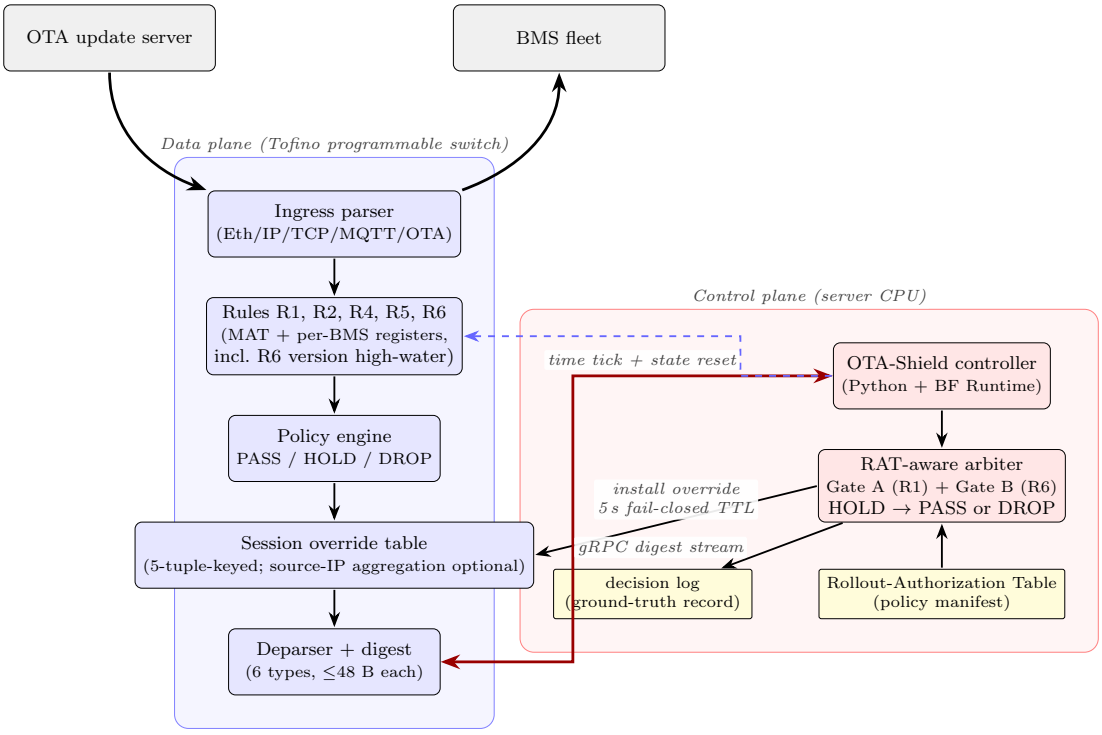


Figure 2: System architecture. The data plane (left) processes every packet through five detection rules (R1, R2, R4, R5, R6), backed by per-BMS state registers including an R6 version high-water mark; digests stream to the control plane (right) via gRPC; the RAT arbiter installs session overrides (5-tuple-keyed by default; a source-IP aggregation variant is preserved as a compile-time knob, see §6.7) with a 5 s fail-closed TTL. Gate A and Gate B (§4.4.1–4.4.2) run inside the arbiter.

Table 3
Who does what: operator, OTA-Shield, and the fleet. OTA-Shield enforces operator-authored policy on the transit path; endpoint signing stays complementary.

Party / component	Responsibility
Vendor / integrator OTA server	Builds and pushes signed firmware images to the fleet.
BESS operator	Authors authorized rollout schedules and rollback windows.
Signed RAT manifest	Encodes the authorized schedule; ed25519-signed by the operator.
OTA-Shield controller	Verifies the signed RAT, installs data-plane tables, arbitrates HOLD events.
Tofino 1 switch	Enforces per-packet and stateful rules at the transit point, at forwarding speed.
RAT arbiter (Gate A / Gate B)	Admits authorized rollouts and rollbacks; holds or denies the rest.
BMS fleet	Receives admitted firmware; endpoint signature verification stays complementary.
Operator / SIEM	Receives audit records of every admit, deny, and HOLD decision.

per-BMS high-water mark and fires when the observed version is strictly lower, which catches a signed-but-older image that endpoint signature verification accepts. R1 (rapid replay) measures the inter-update interval per BMS and fires when it falls below a cadence threshold; it is enabled only when per-BMS cadence is slower than τ_{R1}.

With compile-time thresholds τ_{R1}, τ_{R4}, τ_{R5}, the three threshold-bearing rules fire when

$$\text{R1 fires if } \Delta t_i < \tau_{R1},$$
$$\text{R4 fires if } B_i > \tau_{R4},$$
$$\text{R5 fires if } C_W > \tau_{R5},$$

where Δt_i is the inter-update interval for BMS i, B_i is the session byte count for session i, and C_W is the count of distinct destination BMS addresses inside the R5 window W. R2 fires when the source IP is not in the authorized-source allow-list, and the OTA-header sanity checks are handled by the control plane. R6 fires when the observed OTA-header version v_i is strictly less than the stored per-BMS high-water mark $v_i^\star$. R5's fleet-fanout count is charged only against unauthorized sources, so an authorized fleet rollout does not consume the detector's fanout budget; the implementation gates the count update on the R2 fire flag (§5).

4.4. Policy engine and RAT-aware arbiter

The policy engine splits enforcement into a fast data-plane verdict and a slower control-plane arbitration. In the data plane, the policy control block (Fig. 3) combines per-rule fire flags into a byte-wide action code. R2 (unauthorized source) drops the packet at the deparser. A behavioral rule fire (R1, R4, or R6) sets a `HOLD` action code and emits a `hold_digest` to the controller. The

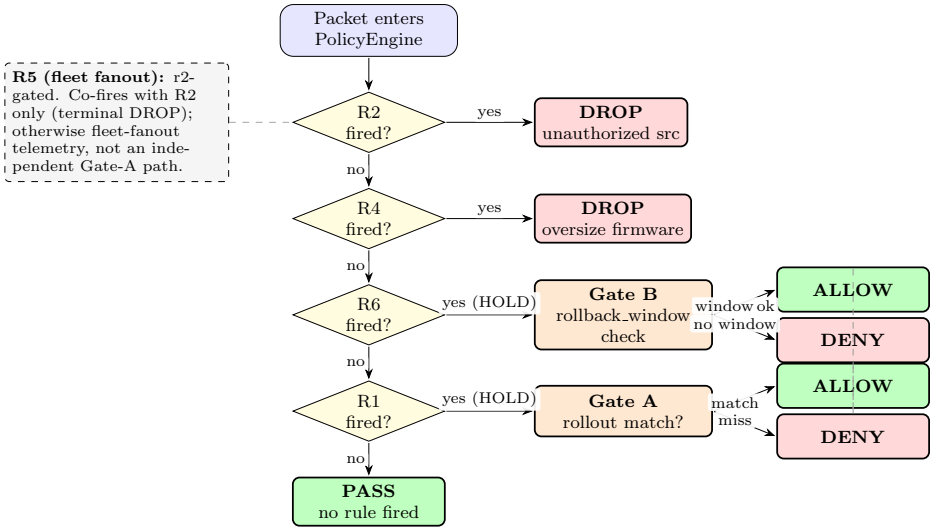


Figure 3: Policy decision flowchart. All rule fires emit a `hold_digest` to the controller. R2 (unauthorized source) and R4 (oversize payload) remain terminal DROP decisions. R1 (rapid replay) and R6 (version rollback) enter control-plane arbitration: Gate A demotes an R1 fire that matches an active authorized rollout to PASS (R5 cannot reach Gate A on the deployed build, its counter being r2-gated, §6.4), and Gate B demotes an R6 fire to PASS only when the active RAT entry carries an explicit `rollback_window` covering the observed version. Any other HOLD becomes a DROP override. Override entries expire after 5 s (fail-closed).

controller then decides: R4 stays terminal, while R1 and R6 go through the two-stage RAT arbiter (§4.4.1–4.4.2), which either installs a `session_allow` override (via Gate A or Gate B) or a `session_deny`. R5 is r2-gated: it co-fires with R2 (terminal DROP) and otherwise contributes a fleet-fanout flag to the digest rather than a distinct arbiter path. Packets that fire no rule pass with action code zero and no digest.

Packets with `action_code` $\neq 0$ generate a `hold_digest` (28 bytes, well under Tofino 1's 48-byte learn quantum) carrying the 5-tuple, session ID, BMS index, all five rule-fire flags, and the OTA version and size. The controller compares this digest against the RAT manifest (Supplementary Table S4); a match installs a `session_allow` override, a miss installs `session_deny`. Both expire via a 5-second timer, giving fail-closed recovery if the controller stalls. Because the digest carries the BMS index and all five rule-fire flags, every detection is localized to a specific device and rule rather than reported only in aggregate, in the manner of localization-aware ICS detectors [**?**]; the per-strategy, per-rule breakdown this enables is reported for the mimicry sweep in §6.5.

4.4.1. Gate A: RAT-gated demotion of R1 fires

Gate A relaxes R1 (rapid replay) for packets that belong to an authorized rollout. Two stages run per HOLD digest. Stage one keeps R2 (unauthorized source) and R4 (oversize payload) terminal: any combination flagged as terminal installs `session_deny`. Stage two looks at R1 fires against the active RAT entries. R5 (fleet fanout) does not reach this gate on the deployed build: the data-plane R5 counter is gated on `r2_fired`, so R5 only fires alongside R2 (an unauthorized source), and stage one has already made that combination terminal.

Notation: r_1, r_2, r_4, r_5, r_6 are the rule-fire flags; s the source IP; d the destination BMS; v the OTA-header version. Gate A demotes an R1-only fire to PASS when $r_2 = r_4 = 0$ and a RAT entry R exists such that $s \in R.authorized\_source\_ips$, $d \in R.target\_bms\_list$, the current time lies inside $R.valid\_window$, the advertised size lies inside $R.expected\_payload\_size\_range$, and v matches $R.expected\_firmware\_version$ (or an immediate neighbour, to tolerate staged major/minor bumps inside one rollout). The controller then installs a `session_allow` override instead of `session_deny`. On the benign workloads of §6.3 and §6.2, Gate A clears inter-update (R1) HOLDs on authorized staged, emergency, and migration rollouts to zero-drop outcomes, with no data-plane threshold retuned.

4.4.2. Gate B: RAT-gated demotion of R6 fires

Gate B relaxes R6 (version rollback) only when the RAT explicitly authorizes the downgrade. R6 follows the same shape as Gate A but through a parallel rollback gate. An R6-only fire ($r_6 = 1$, $r_2 = r_4 = 0$, no other terminal rule co-fired) is demoted to PASS only when a matching RAT entry R exists (same source/destination/window/size checks as Gate A), R carries an explicit `rollback_window` field, and v lies inside the range $[R.min\_version, R.max\_version]$ that the window advertises. If any check fails, R6 stays terminal and the controller installs `session_deny`.

The default is closed. A RAT entry that omits `rollback_window` leaves R6 as a terminal HOLD/-DROP rule, so a deployment that never authorizes a downgrade sees the same attack surface it had before. The gate exists to cover operator-authorized emergency rollbacks (for instance, an OEM recall that moves a fleet from version 48 back to version 47), while the data-plane monotonicity check still fires on every unauthorized downgrade.

Algorithm 1 Two-stage RAT-aware arbiter (Gate A and Gate B).

Require: digest D with flags r_1, r_2, r_4, r_5, r_6, source s, destination d, size b, version v; RAT manifest $\mathcal{R}$.
1: **if** $r_2 = 1$ **or** $r_4 = 1$ **then** ▷ terminal
2: **return** `session_deny`
3: **end if**
4: $R \leftarrow \text{MatchRAT}(\mathcal{R}, s, d, b, now)$
5: **if** $R = \varnothing$ **then**
6: **return** `session_deny`
7: **end if**
8: **if** $r_6 = 1$ **then** ▷ Gate B: default-closed
9: **if** *R.rollback_window* exists **and** $v \in$ *R.rollback_window* **then**
10: **return** `session_allow`
11: **else**
12: **return** `session_deny`
13: **end if**
14: **end if**
15: **if** $r_1 = 1$ **then** ▷ Gate A (r_5 unreachable: gated on r_2)
16: **return** `session_allow`
17: **end if**
18: **return** `session_deny` ▷ unreachable under digest precondition

Gate A handles rollout-context demotion for R1; Gate B handles explicit rollback authorization for R6. MatchRAT checks source, destination, time window, and size range; it does not require exact version equality, which would reject legitimate staged major/minor bumps inside an active campaign. The final `session_deny` is unreachable on well-formed input: the data plane emits `hold_digest` only when $r_1 \vee r_2 \vee r_4 \vee r_5 \vee r_6 = 1$. The guards at lines 2–4 and 9 remove r_2, r_4, and r_6, and Gate A removes r_1. The one remaining case, an r_5-only fire, cannot occur on the gated build: the R5 counter is gated on `r2_fired`, so every r_5 fire also sets r_2 and is already caught by the terminal guard. It is retained as a defensive default for malformed or replayed digests. Every arbiter decision is logged, and all reported classification verdicts are read directly from this decision log.

5. Implementation

This section describes the Tofino 1 realization of the design: the ingress pipeline that evaluates the rules, and the controller that runs the arbiter.

5.1. P4 ingress pipeline

The ingress pipeline comprises a protocol-aware parser and nine MAU-stage control blocks (the data-plane half of Fig. 2), within the 12-stage Tofino 1 budget; topic and OTA header use fixed-size slots (§6.7).

Parser. The ingress parser identifies MQTT traffic on TCP port 1883. For MQTT PUBLISH packets (`mtype`=3), it extracts the variable-length remaining-length field (1–4 bytes), a 32-byte null-padded topic slot, a QoS-1 packet identifier, and the first 20 bytes of the payload as the OTA header. The OTA header carries a 4-byte magic (`0x4F544153` = "OTAS"), a 4-byte firmware version, a 4-byte advertised size, and an 8-byte hash hint. All metadata fields are byte-aligned (`bit<8>`) to satisfy Tofino 1's PHV allocator constraints.

Session manager. A CRC32 hash of the 5-tuple produces a 16-bit session index into two 65 536-entry registers: `session_bytes_reg` (cumulative TCP payload bytes) and `session_first_ts_reg` (first-seen timestamp with a non-zero sentinel). On TCP FIN or RST, a read-and-clear action atomically returns the session summary and resets the slot.

Fleet monitor (R5). Three independent Bloom filters (1024 entries each, CRC32 / CRC16 / CCITT hashes with salt bytes) track distinct destination BMS addresses within a 60-second tumbling window. A single `RegisterAction` on a 16-bit counter increments on Bloom miss and returns the current count. A range-match table fires R5 when the count exceeds the threshold (compile-time constant, default 4). The controller clears all three Bloom registers and the counter every 60 seconds. Bloom inserts and the count update are gated on `r2_fired==1` so authorized fleet rollouts do not consume Bloom slots; only unauthorized fanout charges against the detector. This Bloom-sizing assumption is revisited in §7.2 for fleet-scale deployments.

Secondary rules (R1, R4). R1 uses a 256-slot 16-bit register indexed by BMS number. The stateful ALU computes `coarse_time_sec_lo - last_seen` as the inter-update delta; a range-match table fires R1 when the delta is below the threshold (compile-time constant, derived from the E7 baseline as 14 400 s = 4 h; see §6.2). R4 compares the upper 16 bits of `session_bytes_val` against a range threshold (compile-time constant, derived from E7 as 32 units of 64 KiB = 2 MiB). R2 is an exact-match table on `ipv4.src_addr`: entries for authorized OTA sources are installed by the controller at startup from the RAT manifest. The default action fires R2, implementing a fail-closed policy.

Version rollback (R6). R6 keeps a per-BMS firmware-version high-water mark in a 256-slot 32-bit register indexed by BMS number. The stateful ALU compares the incoming OTA-header version against the stored

mark, fires R6 when the observed version is strictly below it, and raises the slot to the maximum of the two values. There is no threshold to tune: the check is version-monotonic and catches signed-but-older firmware that endpoint signature verification lets through. An R6 fire emits a `hold_digest`, and the control plane decides. Gate B consults the RAT and clears the fire only when an authorized rollback entry is active.

Compile-time thresholds. All thresholds are compile-time constants (range-match table entries) rather than runtime registers, for two reasons. First, a runtime register comparison would need two runtime operands in the same MAU action, which the Tofino 1 gateway's predicate-width constraint disallows. Second, the thresholds are derived once per deployment from an E7-equivalent baseline (§6.2) and are not intended to drift; pinning them at compile time makes their provenance auditable and excludes a silent-override vector on the control plane. Re-deriving thresholds for a new deployment profile is a P4 recompile (minutes), not a runtime reconfiguration.

5.2. Controller

The controller attaches to the switch over local-loopback gRPC. It installs R2 allow-list entries and Bloom-clear timers at startup, receives each `hold_digest`, and runs the two-stage arbiter of §4.4 on every HOLD. A match installs a `session_allow` override on the 5-tuple-keyed `session_action_override` table; a miss installs `session_deny`. Both expire after a 5-second fail-closed TTL. The controller loads the RAT manifest at startup and on hot-reload, verifies its ed25519 signature when run under `-require-signed-rat`, and retains the last-known-good manifest when a reload fails verification.

6. Evaluation

This section answers five research questions about OTA-Shield with the testbed and methodology described in §6.1–6.2.

Main result map. The evaluation supports three claims. *Claim 1, bounded detection works on the designed family*: E8 is the headline result (support: E1, E2, E6). *Claim 2, RAT/R6 admission and rollback handling work*: E12 admits authorized rollouts without false drops and E19/E19′ isolate rollback detection and its first-contact limit (support: E12b, E7b). *Claim 3, the deployment boundaries are measured*: E21, E20, E23, and E18 map the adaptive-mimicry, brokered-MQTT, TCP-segmentation, and parser-variation boundaries (support: E11, E13, E14). E10 positions the design against CPU IDS baselines on identical traffic, showing the contribution is architecture and resource placement, not P4-only detectability. Support and boundary experiments are summarized in Table 6 and discussed in the main text only where they change interpretation; the *Role* column marks each one. Supplementary figures, tables, and methodological detail (per-trial results, the first-contact recall taxonomy, build provenance, statistical methods, and per-component resource placement) are provided in the supplementary material. The coverage map (Table 5) ties each contribution to its evidence and its measured boundary.

- **RQ1.** Does the detector classify attack and benign OTA events correctly on the reference testbed? (E1, E2, E6, E8, E12, E12b.)
- **RQ2.** Is the arbiter necessary for authorized-rollout and rollback admission (as opposed to attack precision), and is R6 the sole detector of unauthorized rollback? (E4, E19, E19′.)
- **RQ3.** How does each rule degrade under adversarial mimicry, threshold sweeps, and long-horizon slow-cadence traffic? (E9, E21, E7b.)
- **RQ4.** How does OTA-Shield compare to CPU-based IDS baselines on identical traffic? (E10 variants i–vii.)
- **RQ5.** What latency, throughput, capacity, and portability costs does the deployment pay? (E14, E11, E13, E18.)

Every result below is measured on one deployed configuration, summarized in Table 4; Table 5 maps each attack pattern to the rule that answers it and to that rule's measured blind zone.

Table 4
Final evaluated build. All results in this section are measured on this single deployed configuration.

Component	Final setting
R5 gating	r2-gated (counts unauthorized-source fanout only)
Override table	5-tuple-keyed default; source-IP aggregation optional
Per-source hold gate	inert (arms on R5 only, which is r2-shadowed)
OTA profile	cleartext MQTT reference profile (TCP/1883)
Parser	32-byte null-padded topic slot + 20-byte OTA header
Fleet	50 BMS emulated endpoints

Table 6 indexes every experiment to the RQ it supports and to the subsection in which it is reported.

6.1. Testbed

The testbed (Fig. 4) consists of two commodity servers bridged by a programmable switch. One server is the OTA update server, publishing MQTT firmware messages. The other emulates a 50-unit BMS fleet using network-layer aliases, so every unit appears on the wire as a distinct endpoint. Both servers carry 25 Gb/s network interfaces and connect to the switch through RS-FEC links. The switch runs BF-SDE 9.13.2, with the OTA-Shield P4 pipeline loaded and the Python controller attached over local-loopback gRPC.

Table 5
OTA-Shield coverage map: how the evaluated build treats each attack pattern and each authorized operation, the mechanism, the primary evidence, and the measured deployment boundary. Rows 1–4 and 7–9 are attacks; rows 5–6 are authorized operations the arbiter admits; rows 8–9 are unsupported boundary cases, not attacks.

Scenario / operation	OTA-Shield behavior	Mechanism	Evidence	Measured boundary
Unauthorized-source OTA campaign	DROP (attack)	R2, with R5 telemetry on unauthorized fanout	E8, E9	brokered-MQTT source-IP collapse (E20)
Oversize firmware image	DROP (attack)	R4 size threshold	E6, E9	2.0–2.0625 MiB quantization dead zone
Rapid replay / re-push	HOLD, then admit or deny	R1 with Gate A	E8, E12	cadence-dependent, non-central
Signed rollback attack	HOLD, then DROP unless authorized	R6 with Gate B	E19, E19′, E7b	first-contact cold start
Authorized rollout	ADMIT	RAT Gate A	E12	RAT correctness and freshness
Authorized rollback	ADMIT only within `rollback_window`	RAT Gate B	E19, E12	`rollback_window` trust surface
Adaptive mimicry	mostly missed	measured blind zone	E21	1.7% coverage
Brokered MQTT	unsupported in this build	source-IP collapse	E20	needs publisher identity in the inspected header
TCP-segmented / encrypted path	unsupported in this build	no parseable OTA header, no reassembly	E23, Fig. 1	needs post-termination plaintext or reassembly

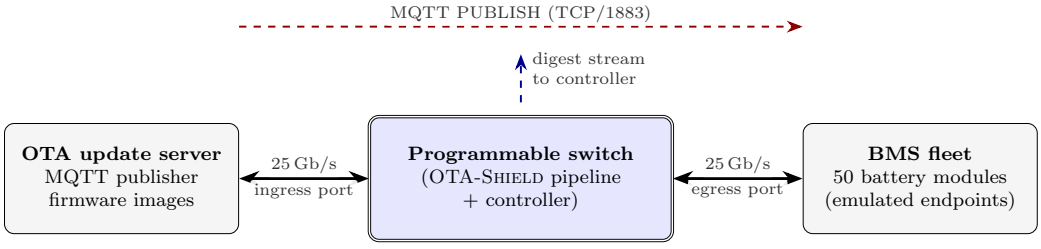


Figure 4: Testbed topology. An OTA update server and a 50-unit BMS fleet connect to a programmable switch through data-plane links configured at 25 Gb/s with RS-FEC (link capability; the experiments in this paper do not offer traffic at that rate). A separate management LAN carries control-plane gRPC traffic.

Traffic generation is scripted: each scenario emits MQTT PUBLISH packets carrying the 20-byte OTA header, records a wall-clock send timestamp and a ground-truth label (LEGIT or ATTACK), and writes the record to a per-trial log. The controller writes every policy decision to a second log. We join the two logs by 4-tuple (`src_ip`, `dst_ip`, `src_port`, `dst_port`) so each labelled event inherits the detector's verdict.

6.2. Methodology

Trial independence. Between trials the sweep driver signals the controller, which re-seeds all R1 last-seen register slots, clears the R5 Bloom filters and counter, and waits 70 s for the R5 window to expire. Each trial starts from identical detector state.

Ground-truth labeling. Labels are assigned by the scenario generator, not by the detector. Events with no matching controller decision or digest are counted as NO_DECISION rather than silently dropped, preventing digest-transport loss from inflating reported metrics.

Threshold derivation versus evaluation (E7). Baseline traffic from the long-run benign trace is used only to derive R5, R1, and R4 thresholds; it is never used to score detection metrics. All reported precision, recall, F_1, and latency numbers come from separately generated evaluation scenarios (E1, E4, E6, E8, E9, E10, E11) that the thresholds have not been exposed to. A 2709-event Poisson baseline trace (≈90 min, mean IAT 2 s) of authorized PUBLISHes reaches random BMSes with firmware sizes drawn uniformly from 256 KiB–2 MiB. From this baseline we compute three distributions: (a) distinct-BMS count per 60-second window (R5 metric), (b) per-BMS inter-rollout interval (R1 metric), and (c) firmware size (R4 metric). The P99.9 tail of the distinct-BMS count is 30.0 (legitimate fleet rollouts reach at most ≈30.0 distinct BMS targets per 60-second window at P99.9), the P0.1 tail of the inter-rollout interval is 0.1 s (R1 minimum interval 1 s), and the P99.9 firmware size is 1995371 B (R4 bound 2095947 B). R4 is pinned directly at the E7-derived ≈2 MiB bound. R5 is a compile-time constant set to 4 (fires when distinct-BMS count exceeds 4 per 60-s window), substantially below E7's P99.9 of 30.0; R5 is intentionally tighter than the

Table 6
What each experiment tests, the claim it supports, and the boundary it measures. The *Role* column tiers each experiment as PRIMARY (a load-bearing claim), BOUNDARY (a scope or blind-zone limit), SUPPORT (correctness, capacity, latency, or cost), or ARTIFACT (manifest-lifecycle and build provenance). Interval methods (bootstrap †, Wilson/Clopper–Pearson ‡, deterministic counts otherwise) are detailed in Supplementary Sec. S4.

Exp.	Purpose	Type	Trials	RQ	Role	Headline
E1	Correctness (attack detection)	deterministic	5	RQ1	Support	$F_1 = 1.000$
E2‡	FP baseline	deterministic	5	RQ1	Support	0 FP
E4	RAT arbiter ablation	deterministic	5	RQ2	Support	$F_1 = 1.000$
E6	Oversized firmware (R4)	deterministic	3	RQ1	Support	$F_1 = 1.000$
E8†	Stochastic IID trials	stochastic	20	RQ1	Primary	$F_1 = 0.996$ [0.993, 0.999]
E9	Adversarial evasion sweeps	deterministic	3×4	RQ3	Boundary	boundary maps
E10	Suricata/Zeek baseline	deterministic	1 (PCAP replay)	RQ4	Primary	see §6.6
E11	Controller digest-channel rate	stress run	1	RQ5	Support	0.09% aggregate loss
E12‡	Benign rollout stress	stress run	3	RQ1	Primary	0 DROP on 450 legit events
E13	Override-table capacity	stress run	1	RQ5	Support	sat. at 4–8 s
E14†	Latency stage breakdown	post-hoc	on E8	RQ5	Support	stage 1 p95 41.8 ms
E21	Per-strategy mimicry	deterministic	30	RQ3	Primary	1.7% coverage (§6.5)
E18	Reference-channel portability	deterministic	1	RQ5	Boundary	4/8 variants parse
E19	R6 rollback ablation	deterministic	3	RQ2	Primary	$F_1 = 1.000$ with R6; collapses without R6 (§6.4)
E12b‡	Signed-manifest fail-safe	stress run	20	RQ1	Artifact	last-known-good (LKG) pass-rate = 100% under sig-t
E7b‡	Long-horizon slow cadence	7.9 h trace	1	RQ3	Support	R6 recall 1.000, precision 1.000 (§6.5)
E23	TCP-segmentation evasion	deterministic	250	RQ3	Boundary	header-split evades all rules (§6.5)
E20	Brokered-MQTT topology	deterministic	80	RQ3	Boundary	IP-collapse drops benign fleet (§7.2)

baseline tail to target attacks reaching $\geq$5 distinct BMS targets, a deliberate design choice for the 50-unit testbed scale, not a threshold derived from the P99.9 tail. R1 is pinned looser (14,400 s vs. E7's P0.1) to reflect the update cadence of scheduled campaigns rather than the minimum legal inter-message gap.

Statistical reporting. OTA-Shield has no learned parameters: the rules are fixed compile-time thresholds and the RAT is operator-supplied, so there is no train/test split to leak across. Deterministic experiments (E1, E4, E5, E6) are correctness tests reported without CIs; stochastic experiments (E8, E19′) report trial-level percentile-bootstrap 95% CIs ($B = 2\,000$). Zero-error cells use a Clopper–Pearson one-sided 95% upper bound (rule-of-three: $\leq 0.50\%$ for E8, $\leq 0.67\%$ for E12). Because the fixed-threshold rules give a degenerate precision–recall sweep (precision stays 1.000), we report operating points on the PR plane (Fig. 7), the appropriate presentation under this class skew [**?**]. For E19′ the between-trial variance lives in recall, where first-contact rollbacks R6 has never seen are reported under both a strict count and a lenient one. Full methodology, the benign-set design that guards against inflated scores [3], and the E19′ recall split are in Supplementary Sec. S4.

6.3. Detection and admission are correct on the reference testbed (RQ1)

Finding: rules fire only where they should, and the arbiter never drops an authorized rollout, including under stochastic perturbation and signed-manifest tampering.

The first block of experiments checks four things: that each rule fires exactly on its designed trigger, that the arbiter returns the intended verdict on every event, that stochastic perturbation does not collapse those guarantees, and that the benign dual (the arbiter must not drop authorized rollouts) holds end-to-end.

Every rule fires on its designed trigger (E1). Each trial sends 30 baseline legitimate PUBLISHes, 30 rapid replays to the same BMSes (R1 fires; the replay source is authorized, so R2 does not fire and R5's r2-gated counter never runs), and 30 unauthorized-source PUBLISHes (R2 fires), 90 events. Across 5 trials, every trial produces precision 1.000, recall 1.000, $F_1 = 1.000$. Each rule fires correctly on its designed trigger and the RAT arbiter classifies every event as intended.

No false positives on authorized traffic (E2). 50 authorized PUBLISHes per trial (all LEGIT), 5 trials. No false positives.

Oversize firmware images are dropped (E6). A single MQTT session pushes 2.125 MiB of TCP payload (above the 2 MiB threshold). R4 fires deterministically on all 3 trials.

Designed-family detection holds under stochastic load (E8). E8 mirrors the E1 scenario but introduces realistic stochasticity: random BMS ordering, Poisson inter-arrival times (mean 100 ms), and ephemeral source ports drawn uniformly from 49152–65535. Each of 20 trials uses a distinct random seed, producing independent draws. Across the 20 trials we observe precision = 1.000 (FP = 0/600 benign events; CP 95% UB on FP rate $\leq$0.50%; bootstrap CI degenerate), recall = 0.992 [0.986, 0.998], $F_1 = 0.996$ [0.993, 0.999] (trial-level percentile bootstrap 95% CIs, $B = 2\,000$ resamples). The 20-trial sample size was chosen so

that the bootstrap CI half-width for F_1 is below 0.01, verified by plotting CI width as a function of trial count from 5 to 20; the width plateaus by trial 15. Per-trial F_1 dispersion is tight (interquartile range within the CI, Supplementary Fig. S1), consistent with the low missed-event rate on this workload; the pooled confusion matrix is Fig. 5. E8's attack mix inherits E1's composition (30 R1 rapid-replay events from the authorized source and 30 R2 unauthorized-source events per trial, plus their benign baseline); R4 oversize, R5 fanout from an unauthorized source, and R6 rollback are exercised in separate experiments (E6, E9, E21, E19). The headline F_1 therefore covers the R1 rapid-replay and R2 unauthorized-source families at stochastic scale on the 50-unit testbed; fleet-scaling precision at 200 BMS is reported separately in S1/S2 (Supplementary Table S6, §6.7).

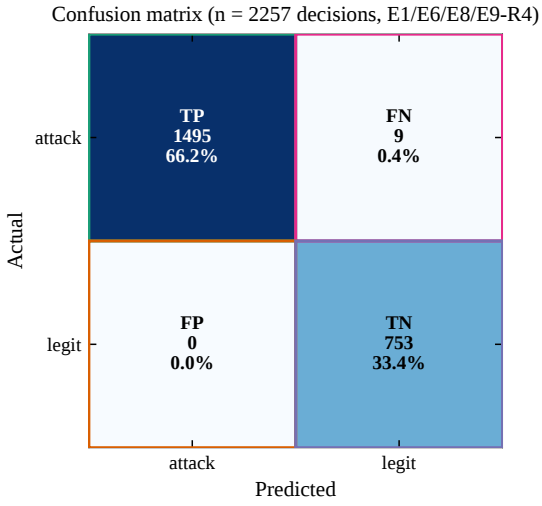


Figure 5: Where the detector's errors fall: pooled confusion matrix over E1, E6, E8, and E9 ($n = 2257$ decisions). Zero false positives and nine false negatives; the nine FNs come from the stochastic trials of E8 and match its recall of 0.992. E4 is an arbiter ablation (RAT disabled), not a detector-scoring run, so it is excluded from the pooled detector matrix.

Authorized rollouts are admitted without false drops (E12). E12 answers the dual of the attack experiments: does the arbiter clear rollouts that an operator is entitled to run? The workload drives six legitimate patterns through Gate A and Gate B (§4.4.1–4.4.2). The backbone is a five-wave staged rollout (ten BMSes per wave, 30 s between waves, 65 s pause to clear the R5 window), and on top of that: an emergency hotfix push that re-hits the same BMS inside R1's 14 400 s window; two migration sub-scenarios that switch the authorized source mid-rollout; a delayed retry burst that compresses a staged wave into a few seconds; a rollback preamble that primes each BMS's version high-water mark; and a benign authorized rollback that downgrades the fleet by one version under an explicit `rollback_window` RAT entry. Every event is labelled LEGIT, so PASS is the correct verdict throughout.

Across 3 trials (Fig. 6) the arbiter produced 0 DROPs on 450 legitimate events (100.00% PASS): 150 silent PASSes (no rule fired) and 300 HOLD-only demotions, the latter being packets that tripped a data-plane rule and were cleared by the control-plane gate. On the deployed gated build the per-rule tally is simple: 270 events fire R1 (a re-push inside R1's window) and are demoted to PASS by Gate A, and 30 authorized rollbacks fire R1+R6 and are demoted by Gate B under an explicit `rollback_window` RAT entry. No legitimate event fires R5 (it is r2-gated, §6.4), so the benign rollouts produce no false positives. The three trials are a reproducibility probe rather than a statistical sample (the scenario is deterministic by construction), so the 0/450 FP count is reported with a Clopper–Pearson one-sided 95% upper bound of 0.66% (at most that fraction of legitimate events could be mislabelled at the 95% confidence level) rather than as an empirical point estimate. No rule-group produced a false positive.

Signed-manifest tampering fails closed (E12b). E12b validates the signed-manifest path end-to-end under controlled tampering. Each of 20 trials has two phases: Phase A runs a benign rollout under a valid ed25519-signed RAT; Phase B injects a single-byte flip on the detached signature file, forcing the controller's hot-reload to reject the update and continue serving the LKG manifest. Across Phase A the arbiter processes 3000 events with 0 DROPs (FP-rate 0.0000); across Phase B the LKG manifest holds 160/160 events at PASS, so the tampered reload never corrupts policy state. The stale-RAT precision failure mode (expired `valid_window`, no replacement installed) is a distinct lifecycle condition discussed in the RAT lifecycle matrix (§7.7).

6.4. The arbiter and R6 are necessary for rollback handling (RQ2)

Finding: the RAT arbiter is necessary for admitting authorized operations, not for attack precision. It clears the authorized rollouts that trip the replay rule R1 (E12, zero benign-rollout false positives) and the operator rollbacks that R6 would otherwise drop (E19, Gate B), and R6 is the sole detector of unauthorized rollback (removing it makes rollback attacks invisible, E19). Disabling the arbiter leaves attack precision unchanged (E4), because the data-plane terminal rules carry attack detection.

RQ2 establishes the arbiter's necessity on the operations it admits, the benign rollouts of E12 (§6.3) and the operator rollbacks of E19 below, and isolates R6 as the sole rollback detector. The attack-side sanity check (E4) comes last.

R5 gating and the deployed-build verdict. On the deployed build the fleet-fanout counter (R5) is gated on

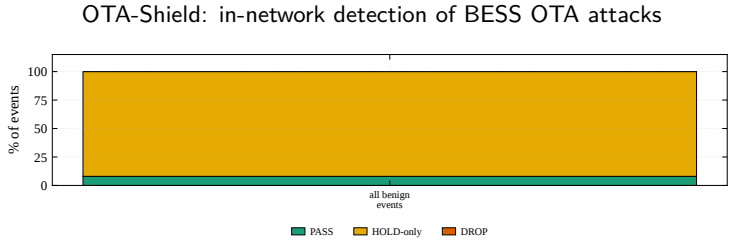


Figure 6: Authorized-rollout admission outcome (E12) across 450 legitimate events (3 trials) on the deployed gated build. Classify-only silent PASS (no rule fired) covers 150 events; the remaining 300 events trip R1 (and R6 on authorized rollbacks) and are demoted to PASS by Gate A (270) or Gate B (30). No legitimate event fires R5: its fanout counter is gated on the unauthorized-source flag (§6.4), so authorized rollouts never reach it. Zero legitimate events are dropped.

`r2_fired`, and R2 is a terminal DROP that the policy engine resolves before the R5 branch. R5 therefore never produces an independent verdict over R2: it contributes fleet-fanout *telemetry* rather than a distinct line-rate action. The per-source `hold_armed_reg` gate arms only on R5, so behind the terminal R2 drop it does not arm on any reachable input and is inert (§7.2). The operational consequence is that an authorized rollout which trips the per-BMS replay rule (R1) or the rollback rule (R6) reaches the arbiter and is admitted, rather than being dropped at line rate before arbitration. The defended space is therefore unauthorized-source campaigns and unauthorized rollbacks, not arbitrary coordinated fanout from an already-authorized source; that blind spot is stated in §7. We keep the gating because it removes benign-rollout false positives, at the cost of R5's marginal independent detection, a trade we judge correct for an OT deployment where a single false quarantine of a legitimate rollout is operationally expensive.

This is the configuration under which E12, E19, and E19′ are measured: E12 (3 trials, 450 legitimate events) reports zero benign-rollout false positives, and E19′ (20 stochastic trials) reports precision 1.000 (Wilson 95% CI $[0.990, 1.000]$) and strict recall 0.857 (Wilson 95% CI $[0.822, 0.886]$). The strict recall counts first-contact rollbacks, which R6 structurally cannot catch because a first contact to a BMS has no stored version high-water mark, as misses (54 of 66 missed attacks); lenient recall excluding them is 0.970. We report the strict figure (rationale in §6.4); R6's first-contact blind spot is a scope limitation addressed in §7, with a baseline-warmup or cross-source version prior as the natural mitigation.

R6 is the sole rollback detector (E19). A targeted ablation confirms that R6 is the sole rollback detector and that disabling it cannot be compensated by R1 alone. With R6 enabled, the arbiter produces 1.000 recall and 1.000 precision over 60 unauthorized rollback events across 3 trials (median end-to-end latency 24.0 ms). With R6 disabled, detection collapses to 0.0%: across 60 rollback ground-truth events in 3 ablation trials, the detector produced 0 DROPs. R1 alone cannot catch a v48→v47 regression because R1 is a sliding-window inter-update test, not a version-monotonicity test; only R6 enforces the high-water mark.

Stochastic rollback (E19′). E19′ runs the unauthorized-rollback experiment with three axes resampled per trial under independent RNG seeds: rollback delta $\sim$ Uniform(9, 16), benign-to-attack event ratio $\sim$ Uniform(0.3, 0.7), and inter-packet gap $= 20$ ms $+ \mathcal{N}(0, 15$ ms) clipped to $[10, 200]$ ms. Across 20 trials on the deployed gated build the detector reports precision = 1.000 [0.990, 1.000] (Wilson 95% CI; zero benign false positives across 340 legitimate events, Clopper–Pearson one-sided 95% upper bound 0.88%) and strict recall = 0.857 [0.822, 0.886], where strict recall counts a first-contact no-decision as a miss. Of the 460 rollback attacks the detector drops 394; the 66 misses split into 12 reachable misses and 54 first-contact rollbacks that R6 structurally cannot catch, because a first contact to a BMS leaves no stored version high-water mark to compare against. Excluding those cold-start cases gives a lenient recall of 0.970 [0.949, 0.983]; we headline the strict figure, in line with evaluation-integrity guidance for security detectors [1, 2]. Because a false quarantine of a legitimate rollout is operationally more costly than a missed cold-start rollback in this setting, we also summarise the operating point with a precision-weighted F_β ($\beta = 0.5$, van Rijsbergen [**?**]): $F_{0.5} = 0.968$ at strict recall and 0.994 at lenient recall. The strict recall reflects R6's first-contact blind spot rather than a detection regression. Controller-side (stage-2) detection latency: 10.7 ms median (p95 15.0 ms); adding the E14 stage-1 measurement ($\approx$29.5 ms) gives end-to-end $\approx$40.2 ms. Per-trial outcomes and the first-contact recall taxonomy are in Supplementary Sec. S2.

Table 7
Rollback detection result (E19′) on the deployed gated build: R6 catches signed-but-older firmware, and first-contact cold starts bound strict recall.

Quantity	Value
Precision (Wilson 95% CI)	1.000 (0 benign FP)
Strict recall (first contact = miss)	0.857
Lenient recall (first contact excluded)	0.970
F_1 (headline, strict)	0.923

The arbiter adds no attack-precision delta on the gated build (E4). For completeness we confirm the converse: the arbiter is not needed for attack precision. Running the E1 workload against a controller launched with `-disable-rat` (which forces every HOLD to DROP) leaves precision at 1.000 and recall at 1.0, a change of Δ precision = 0.0 pp over 5 trials. Every drop is a terminal R1/R2 fire, and no decision is a HOLD the RAT clears, because R5 is gated on `r2_fired` and never charges on an authorized rollout. The arbiter's value is in admission (E12, E19), not attack precision.

6.5. Adversarial robustness and its measured boundaries (RQ3)

Finding: adversaries who shape traffic around each rule's threshold can escape the data plane: 30/1770 (1.7%) of mimicry events are dropped, and the residual blind zones are reported per rule below (Supplementary Fig. S6).

RQ3 asks how each rule degrades when an adversary is free to choose attack parameters near the rule's detection boundary or to slow-play an attack across a realistic deployment horizon.

Evasion sweeps map the rule boundaries (E9). E9 probes the detection boundaries of R1, R4, and R5 by varying the attack parameter across the threshold edge. *R5 sweep:* fanout sizes {3, 4, 5, 6, 7, 10, 15, 20} with disjoint BMS ranges and 65-second pauses; the transition from no-fire to fire occurs at fanout 5, matching the compile-time threshold of 4 (count $> 4 \Rightarrow$ fire). Above the threshold, P(R5 fires) grows as $(N - 4)/N$ (Supplementary Fig. S4), confirming the rule's intrinsic count > 4 boundary. This sweep characterizes R5's counter logic directly; on the deployed build that counter is gated on `r2_fired`, so an *authorized* fleet rollout never increments it and never fires R5 (§6.4). The authorized-rollout PASS rate is therefore 1.0 trivially: there is no R5 fire for the arbiter to demote, not because the arbiter clears an R5 HOLD. *R4 sweep:* flow sizes {1.5, 1.99, 2.0, 2.01, 4.0, 8.0} MiB; R4 fires only at and above 2.0625 MiB, the first size whose upper-16-bit field reaches the range-match threshold (the data-plane compare is `session_bytes[31:16]` $\geq$ 33, i.e. 33×64 KiB). The nominal oversize threshold is 2 MiB (32×64 KiB), but the 64 KiB quantization of the upper-16-bit key means a flow does not trip until it crosses 2.0625 MiB; the 2.0 and 2.01 MiB sweep points fall inside this $[2.0, 2.0625)$ MiB dead zone and do not fire. This is a documented limitation of the range-match encoding. *R1 sweep:* inter-replay intervals {10, 14399, 14400, 14401} seconds; R1 fires at intervals $\leq$ 14399 s and does not fire at $\geq$ 14400 s, confirming the exact boundary. The 16-bit wrap-around creates a theoretical false-positive window at 18.2-hour multiples (§7).

R1 behavior at high traffic rates. Under E7's traffic profile (one PUBLISH per 2 s mean across 50 BMSes), the mean per-BMS revisit interval is approximately 100 s, far below R1's 14 400 s threshold. R1 therefore fires on the vast majority of legitimate baseline packets. This is by design: R1 targets deployments where individual BMSes receive firmware at most once per maintenance window (hours to days), not continuous streaming. The threat model (§2) assumes infrequent OTA updates; E7 demonstrates what happens when that assumption is violated, and its FP rate should be interpreted as a sensitivity bound, not an operational FP measurement.

Adaptive mimicry exposes the main blind zone (E21). E21 probes each rule's documented blind zone with five near-threshold attack strategies, each labelled ATTACK and emitted in its own campaign over 30 repetitions (1770 attack events total). Each strategy is emitted in isolation with a controller state reset between strategies so that the drop, if any, is attributable to the strategy's own target rule; a shared campaign instead lets one strategy's rule fire arm the per-source hold register and cascade drops onto later same-source strategies (§7.2), which inflates the apparent catch rate. The strategies are: (i) *fanout-sub* keeps fanout at exactly 4 per 60 s window across six disjoint BMS groups so R5 does not fire; (ii) *fanout-three* fans a rollout to three BMS, one below the R5 threshold; (iii) *R4 dead-zone* uses payload sizes inside the range-match dead zone so R4 does not fire; (iv) *combined* mixes sub-threshold fanout with the R4 dead zone; (v) *R1 late-replay* re-hits the same BMS with a second replay in each pair.

The per-strategy fire counts and detection rates are in Supplementary Table S9. R5 fires on zero of the 720 fanout-sub and 540 fanout-three events, confirming the sub-threshold-fanout blind zone on the hardware; R4 fires on zero of the 90 dead-zone events, confirming the range-match blind zone. Under per-strategy isolation the only strategy with any detection is *R1-late*, where R1 fires on the second replay of each pair (30/60, 0.50); the *combined* strategy, which a shared continuous campaign had scored as dropped at a usable rate, is in isolation fully evasive (0/360), confirming that the earlier apparent catch was the per-source hold cascade rather than a target-rule fire. Across all strategies the data plane drops only 30 of 1770 attack events (1.7%); the cross-strategy F1 is 0.033 [0.022, 0.049]. The residual 1740 events pass the data plane without a drop. These are the rule set's blind zones under adversarial input design; endpoint signature verification and operator-side rollout monitoring remain needed to close them.

TCP segmentation defeats a header-bound parser (E23). The data plane parses the MQTT/OTA header per packet; it performs no TCP reassembly. We measure the consequence directly with a 5×5 battery: five segmentation evasions against the five header-dependent rules, 10 trials per cell (250 trials), each cell emitted after a controller reset so the outcome is attributable to the cell's own rule. Figure S5 reports the observed

per-cell drop rate. The three *header-split* evasions (`split-mqtt-fh`, `split-topic`, `split-ota-hdr`) cut the single OTA-carrying PUBLISH across two segments so that no segment presents a parseable header; all three evade *every* rule, including the source-keyed R2 (0.00 across the row), because R2's evaluation is gated on the same per-packet OTA parse that the split defeats. The `out-of-order` evasion (halves sent tail-first) likewise evades every parse-dependent rule (R1/R4/R5/R6 = 0.00) while R2 survives (1.00), since its first-sent segment still begins with the MQTT control byte. Only `retransmit-dup`, which leaves an intact PUBLISH on the wire, yields parse-dependent detection, and only probabilistically (R1 0.40, R4 0.60, R6 0.80). No cell fires a rule the threat model did not predict, so no evasion induces a spurious drop. The R5 column is not a faithful fanout test (its single-PUBLISH base attack cannot reach the $\geq$ 4-BMS fanout threshold even un-evaded), so its entries are shown for completeness, not read as fanout-evasion results. The practical consequence: an adversary who fragments the OTA PUBLISH at the TCP layer defeats the header-dependent rules wholesale, so a stateful upstream reassembly point (or a reassembling endpoint check) is required to close this class.

Detection holds over a long slow-cadence trace (E7b). E7b runs the deployed gated build over a realistic deployment horizon: 200 events (100 benign updates plus 100 rollback attacks) over 7.9 h of wall clock, with per-BMS cadence resampled from 18–22 ks (median inter-update interval ≈20,000 s). Two effects show up. First, the slow cadence does not by itself silence the behavioral rules. 20 of the 100 benign updates reach the controller (the other 80 pass natively at the data plane with no decision); R1 fires on all 20 of them, where interleaved re-hits fall inside its window, and the arbiter demotes each to PASS through Gate A. R5 fires on 0 benign updates, since it is r2-gated (§6.4). No benign update is dropped (0 false positives across 100 updates; precision 1.000), consistent with the 0/450 of E12 (§6.3). Second, R6 catches all 100 of 100 rollback attacks (recall 1.000, F1 1.000). Unlike E19', this trace has no first-contact misses: over the long horizon each BMS receives a benign update that establishes its version high-water mark before any rollback targets it, so R6 always has a baseline to compare against. E7b therefore complements E19'. Where E19' isolates the cold-start gap with stochastic first-contact rollbacks (§6.4), E7b shows that once baselines are warm the deployed build sustains $P = R = F_1 = 1.000$ across an 8 h trace, with the arbiter containing every benign rule fire.

6.6. Comparison with CPU-based IDS baselines (RQ4)

Finding: off-the-shelf Suricata and Zeek fire on the fanout shape of legitimate rollouts and cannot suppress it, so they plateau at precision 0.67–0.70 on the same traffic where OTA-Shield reaches 1.00. OTA-Shield's edge is that R5 is gated in the data plane on unauthorized sources, so authorized rollouts never fire it, while the RAT arbiter demotes the R1 re-pushes that do fire. The gap is the policy-aware pipeline, not baseline expressiveness.

Conventional CPU-based IDS, Suricata and Zeek, run on the same port-mirrored traffic a programmable switch can tee, so they are the natural apples-to-apples baseline for an OTA detector that otherwise requires a Tofino 1 pipeline. RQ4 replays one E1 PCAP through five baseline configurations that together span the expressive ceiling reachable without a policy-aware arbiter.

A CPU port matches detection quality (E10). We captured a PCAP of a single E1 trial (90 events: 60 attack + 30 legitimate) and replayed it through four baseline configurations plus one apples-to-apples combination. R6 is omitted from this mapping because the E1 PCAP contains no version-rollback events; the R6 evaluation is reported separately in E19. Supplementary Table S7 collects the six variants; the text below summarizes the arbitration gap they expose.

Variant (i) maps R1, R2, R4, R5 to the closest Suricata 7.0.3 primitive (threshold, content byte match, dsize) and produces zero alerts; it is retained as the expressive-floor anchor. Variant (ii) uses Suricata's richer stateful primitives: `xbits(track ip_dst, expire 14400)` for per-destination replay memory, `flowint` accumulators for byte totals, `threshold by_src count 5, seconds 60` for R5 fanout, a negated allow-list for R2, and `flow:established` to avoid matching synthetic mid-flow packets. Variant (iii) is the same ruleset with `flow:established` relaxed to upper-bound what Suricata rule primitives can express on a synthetic PCAP; it catches every attack but flags every legitimate rollout that trips the same R5 fanout primitive. Variant (iv) replaces the rule DSL with a script-based Zeek detector: a per-destination `last_seen` table (14 400 s expiry) for R1, a per-source fanout set (60 s tumbling window) for R5, an OTAS magic parser for payload inspection, and an R2 authorized-source check. Variant (v) post-processes variant (i)'s Suricata alerts through the same RAT arbiter the OTA-Shield controller runs in-band, giving a rule-DSL detector access to RAT-based demotion. Because variant (i) emits zero alerts on E1, the arbiter receives zero inputs, and the combined system yields recall = 0: giving a rule-DSL detector access to RAT-based demotion does not rescue the expressive floor when the rules themselves never fire. Variant (vi) closes that gap by arbitrating the alerts from variant (iii) (which actually fire) through the same RAT arbiter. Precision climbs to 1.000 (the RAT demotes every false-positive legitimate-rollout alert), but recall drops to 0.500: half the attack events share

the authorized source IP and payload-size envelope of a legitimate rollout, and the RAT arbiter, operating purely on Suricata's 5-tuple + timestamp view, cannot distinguish a rapid-replay attack from a benign rollout on the same flow key without per-flow R1/R4 bits. This is exactly the signal OTA-Shield keeps in the data plane: the arbiter upgrades HOLD → PASS or HOLD → DROP using the rule set that fired on *this* flow, not a post-facto coincidence of 5-tuple and time window. Giving a CPU-IDS baseline the RAT as a black-box post-filter helps precision but silently drops half the attacks.

The pattern across variants (ii)–(iv) is consistent: once the baselines receive the long-horizon state and cardinality primitives needed to catch the attack events, they also flag every legitimate rollout that happens to ride the same primitive. Stateful Suricata and domain-aware Zeek both reach R = 0.78–1.00 but plateau at P = 0.67–0.70 because neither can tell an authorized rollout from a campaign. OTA-Shield avoids these false positives in two ways the baselines cannot: R5 is gated in the data plane on unauthorized sources, so an authorized fleet fanout never fires it, and the RAT arbiter demotes the R1 re-pushes that do fire. This is a policy-awareness gap, not an expressive gap: the detector primitives are reachable on CPU; the data-plane authorization gating and the rule-to-policy arbitration that suppress the legitimate-rollout fires are not. R6 remains inexpressible in Suricata's rule DSL in all variants; the Zeek variant approximates it with the OTAS parser but still cannot distinguish authorized rollbacks from unauthorized ones without RAT-equivalent policy state.

Variant (vii): CPU port of the OTA-Shield rule arbiter. To isolate the in-network performance advantage from the rule expressiveness advantage, we wrote a Python reference implementation of the exact R1–R6 logic and RAT-aware arbitration the data-plane binary executes. On the same 90-event E1 PCAP, the CPU port produces $P = R = F_1 = 1.00$ (TP=60, FP=0, TN=30, FN=0), matching the data-plane binary exactly. Sustained throughput on a single CPU core is $\sim 7 \times 10^5$ packets/s in unoptimized Python on three captures of 90, 4841, and 7310 packets respectively. Compared against the Tofino 1 binary's line-rate forwarding budget on a 25 GbE port ($\sim 3.7 \times 10^7$ pkt/s at 64 B), the CPU port is roughly 50× slower. The expressiveness gap from variants (i)–(vi) collapses to zero once the CPU detector implements the same arbitration we run in the data plane; the remaining gap is performance and resource cost, not detection quality. This separation of concerns motivates §6.7 (latency, throughput, capacity, portability), where the in-network deployment is justified on cost rather than on detection ceiling.

Symmetric comparison on the full E8 population. The E10 baselines above run once on the 90-event E1 PCAP, whereas E8 uses 1800 events over 20 stochastic trials. To match sample size, we reconstructed a per-trial PCAP from each E8 trial and ran Suricata stateful-permissive (best-recall variant) across all 20 trials. The pooled result on 1800 events (P = 0.667, R = 1.000, $F_1 = 0.800$) is identical to the single-trial number, so the asymmetry was sample size, not the conclusion. The 600 false positives are legitimate rollouts Suricata flags on fanout shape and cannot demote; OTA-Shield does not flag them because R5 is r2-gated in the data plane (§6.4), so the precision advantage ($\Delta P = 0.333$, recall gap 0 vs. E8) is a data-plane property, not a control-plane demotion. Gate A's complementary role is the per-BMS replay cohort, where a legitimate re-push trips R1 and the arbiter admits it (E12).

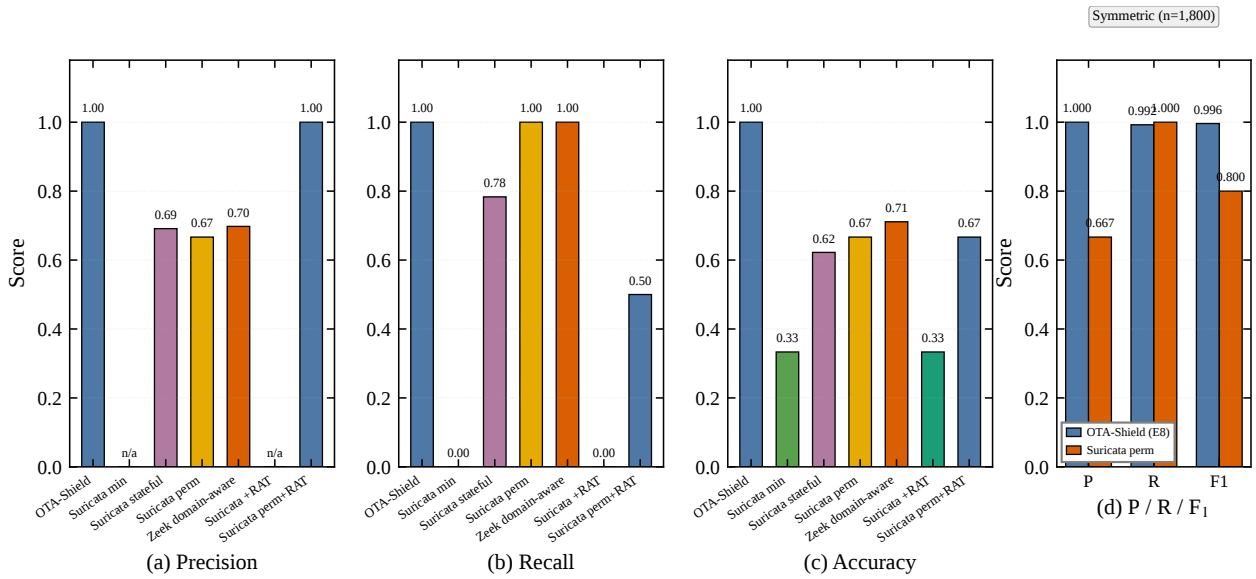
6.7. Latency, capacity, and portability cost (RQ5)

Finding: median latency is 34.0 ms (p95 46.6 ms), the override table holds under sustained install pressure, and 4 of 8 parser variants miss on a single-MQTT-profile sweep. Costs land well inside the operational envelope.

RQ5 reports the cost side of the ledger: end-to-end and per-stage detection latency (E14 on the E8 dataset), controller-side digest ingestion rate (E11), session-override table capacity (E13), and the parser's sensitivity to MQTT-profile variations (E18).

End-to-end latency is dominated by transport, not the ASIC (E14). The two-stage split (Supplementary Fig. S2, medians 34.0 ms end-to-end) shows that *"in-switch"* is not *"in-switch latency"*: stage 1 (generator flight, Tofino 1 pipeline, gRPC transport) dominates the ≈29 ms envelope while the ASIC pipeline is a microsecond-scale term inside it, and the 4.5 ms Python stage 2 (RAT arbitration) is the software-side optimization target. The short tail is two off-path controller events (a state-reset barrier and a timer-based override-TTL sweep), tolerable for an OTA SLA measured in minutes to hours.

The controller sustains the digest channel under offered load (E11). E11 measures the controller's classify/MQTT-parse digest ingestion path (phases 1–2), not the HOLD path: the generator sends non-OTA PUBLISHes to a destination outside the authorized fleet, so no rule fires and no override installs, yet the pipeline still emits one classify/parse digest per packet. Sweeping 100/500/1000/2000 pps, the controller observed 35967/36000 digests, an aggregate loss of 0.09% (worst 0.20% at 100 pps, noise on the smallest window). The scapy generator caps offered load near 2,200 pps, so this is a lower bound on the controller's classify-digest rate, not an upper bound on switch capability or on the HOLD path (exercised separately in E13, where each R5 fire emits one HOLD digest and installs at most one



(a) Precision (b) Recall (c) Accuracy (d) P / R / F_1

Figure 7: CPU-based IDS baselines vs. OTA-Shield. **Panels (a)–(c): asymmetric comparison on the E1 PCAP replay** (90 events: 60 attack + 30 legitimate), seven systems, three metrics. The minimum-effort Suricata mapping and its RAT-arbitrated combination (variant v, apples-to-apples) both floor at recall = 0; stateful Suricata and Zeek domain-aware recover recall to 0.78–1.00 but plateau at precision = 0.67–0.70 because they fire on the shape of legitimate rollouts and cannot suppress it. Variant (vi) = permissive Suricata + RAT reaches precision = 1.00 but its recall drops to 0.500 because RAT without per-flow R1/R4 bits cannot separate same-source rapid-replay attacks from legitimate rollouts. OTA-Shield reaches P = R = 1.00 on the same PCAP. The gap is policy-awareness, not the expressive power of any single primitive: OTA-Shield gates R5 on unauthorized sources in the data plane and demotes R1 re-pushes at the arbiter. **Panel (d): symmetric comparison on the full E8 population** (1800 events across 20 stochastic trials): Suricata stateful-permissive achieves P = 0.667, R = 1.000, F_1 = 0.800; OTA-Shield achieves P = 1.000, R = 0.992, F_1 = 0.996. Precision gap $\Delta P = 0.333$; recall gap = 0.

override per source). This also resolves the "no rule fires ⇒ no digest" phrasing used elsewhere, which referred only to HOLD digests. Line-rate testing of either path needs a DPDK generator such as P4TG [**? ?**] and is left to future work.

The override table has headroom for a 200-source fleet (E13). The deployed `session_action_override` table keys on the full 5-tuple (`src_addr`, `dst_addr`, `dst_port`, `protocol`, `src_port`) at 256 entries, confirmed by `bfrt_grpc` inspection on every T1.7 trial. The 5-tuple key is an isolation property: an `ALLOW` for one flow does not authorize unrelated traffic from the same source. T1.7 installs one `ALLOW` for each of 200 concurrent sources across 20 trials (4000 inserts) with 0 `RESOURCE_EXHAUSTED` rejects (Clopper–Pearson 95% upper bound 13.91%), so the 256-entry table has headroom for a 200-source fleet at one HOLD per source. Because capacity is bounded by 5-tuple cardinality (a single source with churning ephemeral ports generates many 5-tuples), the table is paired with a 5 s fail-closed TTL, and a source-IP aggregation variant is retained for fanout-heavy deployments.

Source-IP aggregation variant. A 1024-entry 5-tuple table saturates within seconds under sustained R5 fanout (a 1000 pps burst against 50 BMSes with unique ephemeral ports floods it before the 5 s TTL evicts). Keying on source IP instead collapses every HOLD from one source into a single entry. Sweeping {1, 10, 32, 64, 100, 200} distinct source IPs at 500 pps (Supplementary Fig. S3), peak active entries track source-IP cardinality exactly (1, 10, 32, 64, 100, 200: 1 entry despite 2051 distinct 5-tuples in the largest window) with 0 rejects across 12476 decisions. Occupancy is thus bounded by source-IP cardinality, typically 1–2 in a BESS fleet. The variant is a compile-time knob; the deployed default is the 5-tuple table characterised by T1.7, chosen for per-flow authorization isolation.

Parser portability is bound to the reference framing (E18). E18 sweeps MQTT-level parameter variations (topic length, QoS level, retain flag) to characterize how tightly the parser is bound to the reference-channel assumptions. This is a scoping result rather than a portability claim: it does not show that the construction ports to an arbitrary MQTT profile, only that deviations from the reference framing require per-channel re-engineering of the parser. Supplementary Table S5 reports the observed pipeline outcome per variant in the core MQTT-parameter sweep.

The deployed parser parses every variant that keeps the 32-byte topic slot (the reference channel and its QoS-2, retain, and QoS-0 variants) and misses every variant that changes the topic length, which shifts the OTA-header offset past the fixed slot. No variant is dropped,

and QoS and the retain flag do not affect parsing. This confirms that the OTA channel used in the evaluation is a reference design rather than a universal model of BESS OTA practice: an operational deployment on a different MQTT profile would require a variable-length topic parser, which would add MAU stages. This is consistent with the scope framing in §1. Under the full 8-variant sweep, the deployed binary parses 4/8 variants end-to-end and misses 4 (all misses driven by topic-length errors); agreement between the predicted and observed per-variant outcome is 7/8. The single disagreement is the QoS=0 variant at the reference 32-byte topic, which parses observably even though the missing 2-byte packet identifier was expected to misalign the OTA header: the deployed parser is more tolerant of the QoS=0 offset shift than the byte layout suggests. The practical implication is that the QoS=0 parser branch prepared for recompile targets the variable-topic-length misses, which are topic-length errors rather than QoS-offset errors. This does not affect the headline portability claim (4/8 parse under the current binary).

Detection holds at 200-BMS scale; precision is cap-limited (S1, S2). The reference numbers reported in §6.3–6.6 are bounded by a 50-BMS testbed. To characterize the design at deployment-scale fleet size, we replay two fleet-scale traces against the live controller. S1 applies a superposed-Poisson workload with 200 synthetic BMS endpoints emitting 4841 events at an aggregate rate of 40 Hz. S2 scales the stochastic E8 mix to the same fleet size, producing 7310 events. Per-BMS rate, payload size, and version-bump probability are inherited from the 50-host long-baseline measurement; only fleet size and the synthetic addresses 10.0.2.60–10.0.2.209 are extrapolated. Before the run, the controller is restarted with the extended manifest so the override table starts empty.

The replay exposes two findings the 50-BMS workload cannot. First, recall is preserved: across both scenarios no rollback attempt is missed (0 and 0 attack-side false negatives), so detection coverage scales linearly with fleet size and the R1/R4/R6 rule chain remains intact under sustained 40 Hz fanout. Second, the controller caps the active BMS-index manifest at 64 entries; the data-plane `bms_ip_to_idx` table itself is compiled at 128 entries (Supplementary Table S3), so the bound here is operational, not a hardware table-size limit. At 200 BMS the manifest cap leaves BMS beyond the first 64 unmapped: a legitimate packet addressed to one of them misses the index table, is signalled `action_code`=2 by the data plane, and lands as a `terminal_fire` DROP at the arbiter. The aggregate result is 0.195 precision and 0.327 F1 on S1 (917 TP, 45 TN, 3781 FP, 0 FN), and 0.292 precision and 0.453 F1 on S2. Digest loss at fleet scale is modest (2.0% on S1, 4.0% on S2), so the digest channel itself is not the bottleneck.

The construction therefore holds at 200-BMS scale on the rule side but not on the table-population side. The 64-entry cap is a deployment-time scope rather than an algorithmic limitation: raising it to the 128 the data-plane table already provides is a controller/manifest change, and covering a fleet beyond 128 additionally requires a P4 recompile to widen the BMS-index, R1, and override tables, after which the same controller and manifest cover the full fleet. We report S1 and S2 alongside the 50-BMS numbers to make explicit which result scales with fleet size (recall) and which depends on a recompile (precision at fleet scale).

7. Deployment Boundaries and Operational Implications

7.1. What the results establish

On the deployed gated build, E4 shows the terminal rules (R2, R4, R6) carry the attack precision without the arbiter: disabling the RAT leaves precision at 1.0, because R5 is gated on `r2_fired` and never fires on an authorized rollout. The arbiter's value is on rollback admission (E19), not an R5 precision delta. The boundary sweeps (E9) map hardware-imposed limits (R4's 64 KiB dead zone between 2.0–2.0625 MiB and R1's 18.2-hour wrap-around) into the threat model. No single rule is evasion-proof; the multi-rule architecture is defense-in-depth within a reference channel. The remainder of this section states the deployment boundaries we measured, and for each names a realistic next step.

7.2. Measured deployment boundaries

The three deployment paths this scope implies were set out early in Fig. 1 (§2): a supported cleartext segment after TLS/VPN termination, and two paths the evaluated build does not cover, a brokered path (E20) and an encrypted or TCP-segmented path (E23). We group the measured limitations into placement, parser and protocol, adversary-adaptation, scaling and capacity, and artifact and reproducibility limits, and for each name a realistic next step.

Placement limits: TLS/VPN scope. OTA-Shield requires placement after TLS/VPN termination or access to plaintext OTA metadata. The R1, R4, R5, and R6 rules all read fields that the parser extracts from the cleartext MQTT PUBLISH and the 20-byte OTA header; on a flow still inside a TLS or VPN tunnel at the switch, the parser cannot recover any of those fields and none of the rules can fire. The deployment requirement is therefore that an upstream broker, IIoT gateway, or VPN concentrator terminate the encrypted leg, and that OTA-Shield sit on the cleartext segment between the terminator and the BMS fleet. Next step: a cross-protocol side-effect rule (R7) correlating post-flash bursts in NTP, DNS, and DHCP, which would cover encrypted OTA channels whose payload cannot be parsed.

Artifact and reproducibility limits: comparative-evaluation scope. Our baseline comparison spans two tiers run on our own traffic, off-the-shelf signature IDS (Suricata, Zeek) and a CPU port of OTA-Shield's own rule and arbiter logic (E10), plus a cited-numbers table positioning OTA-Shield against prior in-network and ICS detectors. We did not run a prior ICS-specific IDS on our own PCAPs. The closest neighbour, a P4-MQTT topic-enforcement detector [**?**], targets broker ACL violations rather than OTA fleet coordination and runs on a software (BMv2) target, so a like-for-like port onto the E1/E8 captures would convert that prose positioning into a measured point; we leave that port to future work. The hardware experiments require an Intel Tofino 1 with BF-SDE 9.13.2 and the emulated BMS testbed: the released artifact ships the P4 source, controller, and result aggregators, but the multi-hour raw run logs and the live testbed are not redistributable, so hardware-bound results are inspectable through the aggregation path rather than re-runnable from the public repository.

Parser and protocol limits. The parser is bound to the reference MQTT framing measured in E18: of the eight QoS and topic-length variants exercised, 4 parse end-to-end and the remainder miss on the fixed 32-byte topic-slot assumption. TCP segmentation (E23, §6.5) splits the OTA header across segments so that no single packet carries the parseable fields, and the data plane performs no reassembly, so a header-split attack evades all rules. Variable-length topic parsing and a reassembly shim are the corresponding extensions.

Placement limits: brokered-MQTT source-IP collapse (E20). Where the OTA path traverses an MQTT broker rather than reaching the fleet directly, a topology some grid profiles mandate, every PUBLISH on the BMS-facing wire carries the broker's source IP. We emulate this on the testbed (80 trials × 30 publishes across one benign and three attack scenarios, with post-relay packets carrying the broker source IP) and the controller fires R2 (unauthorized source) on all 600 of every scenario's events, benign and malicious alike, dropping the benign broker-relayed rollout in full (600/600 false positives). The source-IP-keyed rules cannot separate authorized from rogue publishers once the broker collapses their addresses, so an IP-only authorization table yields no precision in this topology (negative control: precision collapses, F1 undefined under all-drop). The mitigation is to key authorization on a publisher identity carried end-to-end through the broker rather than on the source IP; the data-plane header reserves a 64-bit hash-hint field for exactly this purpose, but the controller-side publisher-id keying is not implemented in the evaluated build. Brokered deployment is therefore a documented gap: we report no publisher-id F1, because with no publisher-id logic in the controller any such number would reflect the IP-keyed all-drop behavior rather than a working publisher-id RAT.

Adversary-adaptation limits: RAT trust surface. Detection accuracy rests on a correct RAT manifest. An attacker with write access before controller start, or a privileged insider authoring entries, can widen a rollback window and suppress R6 for that scope. Because Gate B matches on (*src*, *dst*, *size*, *version*) rather than firmware content, a compromised authorized source can also inject an image whose declared version falls inside a legitimate window, and a sub-threshold fanout (≤ 4, which E21 shows fires R5 on 0/1260 events) carrying such a version passes both gates without ever raising a HOLD. Against an adaptive adversary who reshapes cadence and volume, held-out mimicry coverage falls to 1.7% of attack events (E21). The mitigations are operational: short-lived per-rollout entries, a signed manifest (the controller verifies an ed25519 signature at load and hot-reload and retains the last-known-good cache on failure, E12b Phase B), bounded `rollback_window` spans, and a WARNING-level audit record on every Gate B demotion. Signature verification does not catch replay of an older valid manifest; binding manifests into a monotonic chain is left to operator integration.

Scaling and capacity limits: per-source HOLD gate. The `hold_armed_reg` gate is indexed by the low 8 bits of a CRC16 of the source IP (256 buckets). On the deployed build it arms only on R5, and because R5's counter is gated on `r2_fired` behind the terminal R2 drop, the gate does not arm on any reachable input and is inert; this is why E12 and E19′ report zero benign false positives. (An earlier build armed the gate on any rule fire, which is the configuration recorded in Supplementary Sec. S3.) The 256-bucket index remains a latent limitation for any future build that re-arms the gate on unauthorized fanout: at 10 to 50 concurrent sources, birthday collisions would make it unreliable as a per-source discriminator, and widening the index to a 32-bit CRC32 over the 5-tuple is the mitigation. Rollback detection (R6) further assumes a prior reference version for the targeted BMS: an attack to a device with no baseline yields a no-decision, so lenient recall is 0.970 over devices with an established baseline and strict recall is 0.857 counting first-contact no-decisions as misses (§6.4, Supplementary Sec. S2).

Scaling and capacity limits: Bloom sizing for fleet fanout. The R5 fanout detector indexes each destination into three 1024-entry Bloom filters. Because it gates on `r2_fired==1` (§5), the load that matters is unauthorized fanout per 60-s window, not fleet size, and the deployed sizing is collision-free for the fleet sizes evaluated here. Sustained unauthorized fanout above that band would need wider registers; a 4096-entry resize fits in SRAM without an added MAU stage.

Table 8
Measured deployment boundaries and their mitigations. Items outside the RAT trust surface. Each row names a limitation, its blind zone or failure mode, and an available mitigation path.

Limitation	Blind zone / failure mode	Mitigation
Simulation conventions	Parser assumes 32-byte null-padded topics and QoS = 1 with packet identifier; 4 of 8 swept variants miss the OTA header (E18).	Recirculation or a variable-length extraction pipeline; deployment-specific parser re-engineering per channel.
Switch line-rate throughput	Measured ceiling is ≈2,200 pps on a scapy generator; Tofino 1 line-rate is not exercised. Headline F_1 is bound to the generator, not to the ASIC.	DPDK-class traffic generator (P4TG) on dedicated ports, deferred to future hardware work.
Testbed fidelity	50 BMS endpoints share a single kernel, NIC, and clock; device-side jitter, clock skew, and per-device TCP behavior are not reproduced.	Evaluation on physical-device fleets; timing distributions not measured here.
Session hash collisions	17-bit CRC32 index has birthday-bound collisions ≈ $N(N-1)/2^{17}$; R4 can false-positive or false-negative under concurrent flows. E5′ confirms the analytical bound at $N{=}10\,000$ with 35 observed collisions (1.40%, vs. predicted 1.88%).	Wider index (20-bit, 1 M entries) or set-associative table, at the cost of register memory.
R1 cadence gating	R1 applies only when per-BMS cadence is slower than $\tau_{\mathrm{R1}} = 14\,400$ s; under faster cadences R1 fires on legitimate traffic (E7) and must be disabled.	R2, R4, R5, R6 carry detection under faster cadences; control-plane replay detection for streaming cadences.
Threshold robustness	R5, R1, R4 thresholds are compile-time constants; no runtime adaptive thresholds.	By design: compile-time constants keep thresholds auditable and immutable; per-deployment recompile using E7 percentiles and E9 sensitivity.
Parser short-packet DoS	PUBLISHes with payload <20 bytes to OTA topics trigger a parser error and drop: fail-closed for attacks, but also drops legitimate short status messages.	Topic-prefix ACL upstream of the parser to scope fail-closed behavior.
Override table capacity	`session_action_override` keys on the full 5-tuple (compile size 256); occupancy bounded by 5-tuple cardinality under ephemeral-port churn (T1.7: 4000 inserts across 200 sources, 0 RESOURCE_EXHAUSTED).	Compile-time size is the knob; a source-IP aggregation variant (occupancy bounded by source-IP cardinality, E13) is preserved for fanout-heavy deployments.

The remaining limitations, including the session-hash collision bound (E5′), the cadence-gated status of R1 (E7), and the override-table capacity evidence (E13), are summarised in Table 8.

7.3. Positioning relative to prior systems

No prior system addresses the same combination of BESS OTA semantics, in-pipeline data-plane enforcement, and policy-aware control-plane arbitration, so numeric comparison carries caveats. Table 9 places OTA-Shield alongside the closest prior work by detection domain, platform, and headline metric. Each row reports a number from the cited paper on that paper's own dataset; they are not numbers from a common benchmark, and no single row is directly comparable to ours.

On the same PCAP we benchmarked four CPU-IDS configurations (§6.6): a minimum-effort Suricata mapping (recall 0.000), a stateful Suricata mapping using `xbits`/`flowint`/`threshold` (P = 0.691, R = 0.783), the same mapping with `flow:established` relaxed as an expressive upper bound (P = 0.667, R = 1.000), and a Zeek domain-aware script-based detector (P = 0.698, R = 1.000). Variants (ii)–(iv) all cluster at P ≈ 0.67–0.70 once they reach useful recall. The bottleneck on a CPU IDS is not data-plane line rate or detector expressiveness; it is that none of the CPU baselines can

Table 9
How OTA-Shield compares with related in-network detectors and CPU-based IDS. Each row reports a metric from the cited paper on *that paper's own dataset*; the dataset column makes explicit that these are not measured on a shared benchmark, and the numbers are not comparable across rows. The OTA-Shield row and the E10 CPU-IDS rows are the only rows measured on the same traffic as this paper.

System	Platform	Domain	Result on cited dataset (not comparable across rows)
Raj & Jin [?]	Tofino	Dynamic-data ML IDS	CICIDS: high accuracy
Chen et al. [?]	Tofino	P4 NIDS (distilled trees)	authors' trace: low-SRAM pipeline
AlSabeh et al. [?]	P4 sw.	DDoS entropy DPI	authors' DDoS trace: in-paper metric
Jaqen [?]	Tofino	Volumetric DDoS	authors' testbed: 380 Gb/s mitigation
MQTT pipeline [?]	BMv2 (soft)	MQTT enforcement	authors' emulator: 99.8%
Measured on the E10 PCAP in this paper:			
Suricata 7 (stateful) [?]	CPU	Signature IDS + xbits/flowint	E10 PCAP: P=0.69, R=0.78
Zeek (domain-aware)	CPU	Script-based IDS	E10 PCAP: P=0.70, R=1.00
OTA-Shield (this work)	Tofino 1	BESS OTA fleet attacks	E8 PCAP: F_1 0.996 [0.993, 0.999]

tell an authorized rollout from a coordinated campaign. OTA-Shield gets this from two policy-aware mechanisms the baselines lack: R5's fanout counter is gated in the data plane on the unauthorized-source flag, so authorized rollouts never trip it, and the RAT arbiter demotes the legitimate re-pushes that do fire R1. Its precision advantage at matched recall is therefore a property of the policy-aware pipeline as a whole, chiefly the data-plane r2-gating of R5 on this fanout workload, rather than of the control-plane arbiter alone.

7.4. Operational implications

For BESS operators, OTA-Shield provides a detection layer that operates at the network aggregation point (between the OTA server and the BMS fleet), where coordinated campaigns are visible but endpoint protections are blind. The RAT enables scheduled rollouts to proceed without false alarms, while the 5-second fail-closed override window limits the exposure of any single detection miss. The compile-time threshold constants require a P4 recompile to change (minutes, not hours), which is consistent with planned maintenance cycles but not with real-time threshold adaptation.

7.5. Composing OTA-Shield with endpoint and operator-side defenses

OTA-Shield is one layer in a defense-in-depth stack, not a standalone detector. Supplementary Table S8 sketches what each layer catches and misses.

7.6. Resource and deployment cost

Unlike CPU-based IDS, the data-plane layer pays its cost in MAU stages, SRAM, and amortized switch hardware rather than in cores and watts. Supplementary Table S10 collects the measured resource footprint.

Supplementary Table S3 gives the per-component placement extracted from the deployed build's compiler output. OTA-Shield uses all 12 ingress MAU stages; the long pole is the chain of dependent rule state (session bytes → R4/R5 thresholds → R1/R6 history → policy → override), each link of which must follow its producer's stage. The three range-match threshold checks (R1, R4, R5) account for the TCAM use, and the stateful registers account for the SALU use: the three 1024-entry R5 Bloom filters and the two 64 K-entry session arrays. The full stage occupancy is the binding constraint on adding rules: a seventh behavioral rule with its own stage-dependent state would not fit without folding existing logic, which is one reason new detection logic is added in the controller rather than the data plane. (We also note that `bms_ip_to_idx` is sized at 128 entries in the compiled table; the 64-BMS figure quoted elsewhere is a controller/manifest-imposed cap, not the data-plane table size.)

Two caveats on the cost table. The switch cost is amortized over a single 50-unit fleet; in a larger substation with multiple fleets sharing the switch, the per-BMS figure falls proportionally. The power comparison is approximate: Tofino 1 ASIC power is roughly constant (the pipeline cost is marginal once the ASIC is on), while CPU IDS power scales with offered load.

At the 50-unit fleet measured here, Suricata is roughly 2× cheaper in $/BMS-year at list prices; we report Supplementary Table S10 as a direct measurement at our evaluated scale rather than as a claim that OTA-Shield is cheaper per BMS in absolute terms. The in-network deployment contributes performance headroom and co-location with the forwarding path, not cost or unique detection capability at this fleet size. Two regimes where the tradeoff may favor in-network placement are: (a) multi-fleet substations where the switch is amortized across hundreds of BMS (e.g., a 200 + BMS substation, lowering the per-BMS cost to ≈$20); and (b) high-OTA-traffic loads where a CPU-bound IDS would need per-packet reassembly at rates the ASIC handles with zero incremental cost. Serving such a fleet means raising the 64-entry BMS manifest cap (a controller change up to the 128 the table already holds) and, beyond 128 BMS, a per-deployment P4 recompile to widen the index; without that headroom precision collapses as measured in S1 (§6.7). Quantifying

the cost-parity crossover point is deferred to future work with a DPDK-class traffic generator.

7.7. RAT manifest lifecycle

The RAT is generated by the operator's deployment system (Ansible, a CI pipeline, or a vendor portal), signed, and hot-reloaded into the controller (inotify, with a 5-second polling fallback). E12b confirms the reload is fail-safe: a single-byte signature flip is rejected, last-known-good policy holds across 160/160 post-tamper events with 0 false positives, and the controller never falls through to unsigned mode (20/20 trials). A stale manifest is a pure-loss failure: genuine rollouts fall outside the active window and their R1 fires become DROP overrides, so freshness is an operator responsibility. On the deployed gated build the active-authorized lifecycle state (E22) reproduces E12 exactly: 80 authorized events PASS with zero false positives. The other matrix states probe enforcement the data plane does not perform, since off-parameter operation from an already-authorized source is admitted by design (the documented R5-gating scope, §6.4). Rollback decisions ride the same signed path: Gate B (§4.4.2) expresses an authorized downgrade as an optional `rollback_window`; without it R6 stays terminal, and a stale manifest degrades to R6-as-terminal rather than silent suppression.

8. Conclusion

We described OTA-Shield, a hardware-measured reference architecture for in-network admission of authorized OTA rollouts and bounded detection of OTA firmware attacks against battery energy storage systems, implemented on an Intel Tofino 1 programmable switch. The design places an OTA parser and fleet-cardinality counter on the transit path where endpoint signature checks are blind, and a policy-aware control-plane arbiter (Gate A, Gate B) that consults a Rollout-Authorization Table on a cleartext segment after TLS or VPN termination.

On a 50-unit emulated BMS testbed the data-plane rules hold attack precision at 1.0 without control-plane help (E4 ablation on the gated build); the RAT arbiter's measured role is admitting operator-authorized rollbacks that the version-monotonicity rule (R6) would otherwise drop (E19). The held-out mimicry study reports each rule's measured blind zone (adaptive mimicry drops coverage to 1.7% of attack events), and the capacity sweep reports when the session-override table saturates under sustained install pressure. The design also detects signed rollback attempts through a per-BMS version-monotonicity rule (R6), with authorized downgrade exceptions handled explicitly through Gate B.

The contribution is the arbitration architecture: a data-plane rule set gated so that authorized rollouts never reach the fanout counter, and a RAT arbiter that admits the operator rollbacks the version rule would otherwise drop. A second contribution is the deployment-boundary characterization: the evaluation measures where the architecture holds and where brokered MQTT, TCP segmentation, parser variation, and adaptive mimicry require an extension. Two scopes bound the result. The $F_1 = 0.996$ headline covers the designed fleet-fanout family; an adversary who reshapes cadence faces the 1.7% mimicry coverage instead. Switch line-rate is not measured here, and the end-to-end F_1 is bound by a scapy generator ceiling of $\approx$2,200 pps. The reference-channel profile, the cadence-gated R1, and the bounded Suricata comparison are detailed in §7. Future work includes variable-length topic parsing, a set-associative session table for larger fleets, held-out OTA-channel variants (binary-diff firmware, broker-relayed MQTT, non-MQTT transports), and operational integration with complementary telemetry monitoring. A cross-protocol side-effect rule (R7) that correlates post-flash bursts in NTP, DNS, and DHCP within a short window is a further extension that would cover encrypted OTA channels whose payload we cannot parse.

CRediT authorship contribution statement

Philip Akekudaga: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Writing – original draft, Writing – review & editing.

Data availability

The P4 source, controller, and evaluation aggregators are openly available (Akekudaga, 2026) at `https://github.com/akekulip/OTA-Shield`. The multi-hour hardware run logs and the live testbed are not redistributable; the artifact README documents what can be reproduced.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude (Anthropic) to assist with code drafting and with language editing (grammar and phrasing). After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.